Fundraising Platform — Complete SQL Server Implementation Blueprint

Complete SQL Server implementation blueprint • FundraisingDb • PaymentDb • ReportingDb • performance • concurrency • security • administration

What This Document Is For

This is the **SQL Server implementation document** for the three-database fundraising platform.

The three databases are:

EXAMPLE / FLOW

```
FundraisingDb
PaymentDb
ReportingDb
```

Their responsibilities remain:

EXAMPLE / FLOW

```
FundraisingDb
→ run the fundraising business

PaymentDb
→ process and reconcile money

ReportingDb
→ analyze fundraising and payment data
```

This document covers the SQL Server side from beginning to end:

EXAMPLE / FLOW

```
Database creation
↓
Schemas
↓
Tables
↓
Data types
↓
Primary / Foreign / Unique keys
↓
Constraints
↓
Relationships
↓
Indexes
↓
CRUD
↓
```

```
Joins / Queries
↓
Views
↓
Functions
↓
Stored Procedures
↓
Transactions
↓
Error Handling
↓
Pagination
↓
Temporary Data
↓
Window Functions
↓
JSON / XML
↓
Temporal Tables
↓
CDC / Change Tracking
↓
Outbox
↓
OLTP → ReportingDb ETL
↓
Execution Plans
↓
Query Performance
↓
Concurrency / Locking
↓
Security
↓
Backup / Recovery
↓
Integrity / Maintenance
↓
High Availability concepts
```

It also maps the database work to the **future .NET API endpoints**.

Important:

> SQL Server itself does not expose HTTP endpoints such as `GET /api/gifts`. The endpoint will exist later in ASP.NET Core. In this document, an endpoint is shown so you know **which SQL table, view, query, or stored procedure the future API will use**.

For example:

EXAMPLE / FLOW

```
Future HTTP endpoint
POST /api/gifts
        |
        v
ASP.NET Core later
        |
        v
SQL Server
fundraising.usp_CreateGift
        |
        v
Gift
GiftAllocation
OutboxMessage
```

That is the connection between what you are learning now and what you will build later.

✦ Technology Baseline

Use:

EXAMPLE / FLOW

```
SQL Server 2025
Database compatibility level 170
SSMS
```

for the learning project unless you deliberately test older compatibility behavior.

SQL Server 2025 adds useful modern capabilities such as the native `json` data type, Optional Parameter Plan Optimization, and optimized locking. Treat these as modern enhancements rather than excuses to ignore good schema, query, indexing, and transaction design.

✦ The Most Important Architecture Rules

Before creating anything, keep these rules fixed.

Each database owns its data

EXAMPLE / FLOW

```
FundraisingDb owns
→ Tenant
→ AppUser
→ Constituent
→ Campaign
→ Fund
→ Gift
→ Interaction
→ Task

PaymentDb owns
→ Payment
→ PaymentAttempt
→ Refund
→ Settlement
→ payment processing state

ReportingDb owns
→ dimensions
→ facts
→ analytical representation
```

Three-Database Responsibility Model

FundraisingDb
CRM • Gifts • Campaigns • Engagement • Outbox
Run the fundraising business

→

PaymentDb
Payments • Attempts • Refunds • Webhooks • Settlement
Process and reconcile money

→

ReportingDb
Dimensions • Facts • Historical analytics
Analyze fundraising and payment data

Database boundaries communicate through logical identifiers, events, CDC/ETL, background processing, and eventual consistency - not cross-database foreign keys or one giant business transaction.

No cross-database foreign keys

PaymentDb may contain:

EXAMPLE / FLOW

```
FundraisingGiftId
```

but do not create:

SQL

```
FOREIGN KEY
REFERENCES FundraisingDb.fundraising.Gift(GiftId);
```

Why?

Because later:

EXAMPLE / FLOW

```
Fundraising
```

and:

EXAMPLE / FLOW

```
Payments
```

may become separate services.

Use:

EXAMPLE / FLOW

```
logical identifiers
events
application validation
eventual consistency
```

between database boundaries.

Transactions stay inside one service/database boundary

Correct:

EXAMPLE / FLOW

```
FundraisingDb transaction

Gift
+
GiftAllocation
+
OutboxMessage
+
COMMIT
```

Incorrect architectural direction:

EXAMPLE / FLOW

```
One transaction
    |
    + FundraisingDb
    + PaymentDb
    + ReportingDb
```

Reporting and payment synchronization should happen through:

EXAMPLE / FLOW

```
events
CDC
ETL
background processing
```

later.

✦ SQL Project Folder Structure

Do not create one giant `.sql` file.

Organize your scripts.

EXAMPLE / FLOW

```
database/
│
├── FundraisingDb/
│   ├── 00-Database/
│   ├── 01-Schemas/
│   ├── 02-Tables/
│   ├── 03-Constraints/
│   ├── 04-Indexes/
│   ├── 05-Views/
│   ├── 06-Functions/
│   ├── 07-Procedures/
│   ├── 08-Security/
│   ├── 09-Seed/
│   ├── 10-Reports/
│   ├── 11-Performance/
│   ├── 12-ConcurrencyLabs/
│   ├── 13-AdvancedFeatures/
│   └── 14-Tests/
│
├── PaymentDb/
│   └── same general structure
│
├── ReportingDb/
│   └── same general structure
│
├── Admin/
│   ├── Backup/
│   ├── Restore/
│   ├── Integrity/
│   ├── Maintenance/
│   └── Monitoring/
│
└── README.md
```

The order matters because later objects depend on earlier objects.

For example:

EXAMPLE / FLOW

```
Database
    ↓
Schema
    ↓
Table
    ↓
Constraint
    ↓
Index
    ↓
View / Function
    ↓
Stored Procedure
    ↓
Security
```

✦ Database Creation

Create the three databases separately.

SQL

```sql
CREATE DATABASE FundraisingDb;
GO

CREATE DATABASE PaymentDb;
GO

CREATE DATABASE ReportingDb;
GO
```

For learning, this simple creation is enough initially.

Later you should also study:

EXAMPLE / FLOW

```
Data files
Log files
Filegroups
Initial size
Autogrowth
Recovery model
Compatibility level
Query Store
```

✦ Compatibility Level

For SQL Server 2025:

```sql
ALTER DATABASE FundraisingDb
SET COMPATIBILITY_LEVEL = 170;
GO
```

Repeat appropriately for:

```
PaymentDb
ReportingDb
```

Compatibility level controls optimizer behaviors and enables modern features.

For example, SQL Server 2025's Optional Parameter Plan Optimization requires compatibility level 170 and is enabled by default under the normal SQL Server 2025 configuration.

✦ Enable Query Store Early

Use Query Store while developing so you build the habit of looking at historical query performance.

Example:

```sql
ALTER DATABASE FundraisingDb
SET QUERY_STORE = ON;
GO
```

Do the same for:

```
PaymentDb
ReportingDb
```

Query Store helps you later answer:

```
Which queries are slow?

Did the execution plan change?

Did CPU increase?

Did a deployment create a regression?

How many times does this query run?
```

SQL Server 2022 and later enable Query Store by default for newly created databases, but explicitly understanding/configuring it remains important in a learning project.

✦ Database Schemas

FundraisingDb

Create:

```sql
USE FundraisingDb;
GO

CREATE SCHEMA core;
GO

CREATE SCHEMA constituent;
GO

CREATE SCHEMA fundraising;
GO

CREATE SCHEMA engagement;
GO

CREATE SCHEMA integration;
GO

CREATE SCHEMA reporting;
GO

CREATE SCHEMA security;
GO
```

Purpose:

EXAMPLE / FLOW

```
core
→ Tenant and application-user information

constituent
→ donor/prospect information

fundraising
→ Campaign, Fund, Gift

engagement
→ interactions and Tasks

integration
→ Outbox / integration objects

reporting
→ database-local reporting views/procedures

security
→ RLS predicate functions/policies
```

✦ PaymentDb

SQL

```
USE PaymentDb;
GO

CREATE SCHEMA payment;
GO

CREATE SCHEMA integration;
GO

CREATE SCHEMA reporting;
GO

CREATE SCHEMA security;
GO
```

✦ ReportingDb

A simple warehouse can initially use:

EXAMPLE / FLOW

```
dbo
```

for its dimensions/facts.

Later you could separate:

EXAMPLE / FLOW

```
dimension
fact
etl
```

but do not create complexity merely for appearance.

✦ Choosing SQL Server Data Types

Do not choose types randomly.

The type should model the business value.

BIGINT

Use for high-growth operational identifiers.

Examples:

EXAMPLE / FLOW

```
GiftId
PaymentId
ConstituentId
InteractionId
```

Why?

A busy long-lived platform can generate many records.

✦ INT

Good for warehouse surrogate keys and smaller enumerations where billions of values are unnecessary.

Example:

EXAMPLE / FLOW

```
TenantKey
CampaignKey
DateKey
```

✦ DECIMAL(19,4)

Use for money in this project.

Example:

EXAMPLE / FLOW

```
TotalAmount
AllocationAmount
RefundAmount
GrossAmount
FeeAmount
```

Do not use:

EXAMPLE / FLOW

```
FLOAT
REAL
```

for financial amounts.

Why?

Those are approximate numeric types.

Money calculations require predictable exact numeric representation.

✦ CHAR(3)

Good for ISO-style currency codes:

EXAMPLE / FLOW

```
USD
EUR
GBP
```

because they have fixed width.

✦ NVARCHAR

Use when text may contain international characters.

Examples:

EXAMPLE / FLOW

```
DisplayName
CampaignName
Description
Notes
```

✦ VARCHAR

Useful when values are deliberately limited to ASCII-compatible technical/business codes.

Examples:

EXAMPLE / FLOW

```
CampaignCode
ProviderPaymentId
ReferenceNumber
```

✦ DATETIME2(3)

Use for most application timestamps.

Example:

EXAMPLE / FLOW

```
CreatedAtUtc
UpdatedAtUtc
OccurredAtUtc
```

Why `(3)`?

Millisecond precision is normally adequate for business APIs and avoids unnecessary precision.

Use UTC throughout database timestamps:

EXAMPLE / FLOW

```
CreatedAtUtc
```

rather than storing local server time.

✦ DATE

Use when only the business date matters.

Examples:

EXAMPLE / FLOW

```
GiftDate
Campaign.StartDate
Campaign.EndDate
SettlementDate
```

✦ BIT

Use for simple true/false state.

Examples:

EXAMPLE / FLOW

```
IsActive
IsRestricted
IsPrimary
IsCurrent
```

✦ UNIQUEIDENTIFIER

Good for event identifiers and externally safe distributed identifiers.

Example:

EXAMPLE / FLOW

```
OutboxMessageId
```

Do not automatically make every OLTP clustered primary key a random GUID.

Random GUID clustering can create unnecessary index fragmentation/page splits unless the architecture genuinely benefits from it.

✦ TINYINT

Good for simple small status/type columns.

Example:

EXAMPLE / FLOW

```
GiftType

PaymentStatus

InteractionType

Priority
```

This project deliberately does not create a status table for every type.

A `CHECK` constraint is sufficient while the value set is stable and simple.

✦ Keys and Constraints Strategy

Database constraints are not optional decoration.

They protect your database even when:

EXAMPLE / FLOW

```
.NET has a bug

a script is incorrect

another service writes data

a developer makes a mistake
```

Use:

EXAMPLE / FLOW

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
CHECK
DEFAULT
NOT NULL
```

where the business rule belongs in the database.

✦ Primary Keys

Every table needs a stable row identifier.

Example:

SQL

```sql
TenantId BIGINT IDENTITY(1,1)
    CONSTRAINT PK_Tenant PRIMARY KEY
```

The primary key identifies the row.

Do not confuse:

EXAMPLE / FLOW

```
PRIMARY KEY
```

with:

EXAMPLE / FLOW

```
business code
```

For example:

EXAMPLE / FLOW

```
CampaignId
```

may be the PK.

EXAMPLE / FLOW

```
CampaignCode
```

can be a separate unique business value.

✦ Foreign Keys

Foreign keys enforce relationships inside one database boundary.

Example:

SQL

```
CONSTRAINT FK_Gift_Constituent
FOREIGN KEY (ConstituentId)
REFERENCES constituent.Constituent(ConstituentId)
```

Foreign keys prevent:

EXAMPLE / FLOW

```
Gift referencing nonexistent Constituent
```

but remember:

> SQL Server does not automatically create a useful child-side index merely because you created a foreign key.

Create indexes based on actual join/filter workloads.

✦ UNIQUE Constraints / Unique Indexes

Use uniqueness for real business rules.

Examples:

EXAMPLE / FLOW

```
TenantCode

Tenant + AppUser Email

Tenant + CampaignCode

Tenant + FundCode

Provider + ProviderEventId

Tenant + IdempotencyKey
```

✦ CHECK Constraints

Use `CHECK` for simple row-level rules.

Examples:

SQL

```sql
CHECK (TotalAmount > 0)
```

SQL

```sql
CHECK (EndDate IS NULL
       OR StartDate IS NULL
       OR EndDate >= StartDate)
```

SQL

```sql
CHECK (PaymentStatus BETWEEN 1 AND 7)
```

Remember:

A SQL Server `CHECK` constraint passes when the expression is:

EXAMPLE / FLOW

```
TRUE
or UNKNOWN
```

so nullable columns need deliberate design.

✦ DEFAULT Constraints

Use defaults for values such as:

EXAMPLE / FLOW

```
CreatedAtUtc
RetryCount
IsActive
```

Example:

SQL

```
CreatedAtUtc DATETIME2(3)
    NOT NULL
    CONSTRAINT DF_Gift_CreatedAtUtc
    DEFAULT SYSUTCDATETIME()
```

✦ ROWVERSION

Add `ROWVERSION` to frequently edited OLTP entities where you want optimistic concurrency.

Good candidates:

EXAMPLE / FLOW

```
Constituent
Campaign
Fund
Gift
Task
Payment
```

Example:

SQL

```
RowVersion ROWVERSION NOT NULL
```

Later .NET can perform:

SQL

```
UPDATE ...
WHERE GiftId = @GiftId
  AND RowVersion = @OriginalRowVersion;
```

If:

EXAMPLE / FLOW

```
0 rows affected
```

the application knows another transaction modified the row.

Do not use `rowversion` as a date/time.

It is a binary change token.

✦ FundraisingDb Table Design

The database has twelve core tables.

✦ core.Tenant

Purpose:

> One organization using the fundraising platform.

Columns:

EXAMPLE / FLOW

```
TenantId          BIGINT IDENTITY PK
TenantCode        VARCHAR(50) NOT NULL
TenantName        NVARCHAR(200) NOT NULL
CreatedAtUtc      DATETIME2(3) NOT NULL
UpdatedAtUtc      DATETIME2(3) NULL
RowVersion        ROWVERSION
```

Rules:

EXAMPLE / FLOW

```
TenantCode unique
```

Recommended:

SQL

```sql
CREATE UNIQUE INDEX UX_Tenant_TenantCode
ON core.Tenant(TenantCode);
```

Future API mappings:

EXAMPLE / FLOW

```
POST /api/tenants
→ INSERT Tenant

GET /api/tenants/{id}
→ SELECT Tenant

PUT /api/tenants/{id}
→ UPDATE Tenant using RowVersion
```

✦ core.AppUser

Purpose:

> Employee who uses the fundraising application.

Columns:

EXAMPLE / FLOW

```
AppUserId          BIGINT IDENTITY PK
TenantId           BIGINT FK
EntraObjectId      UNIQUEIDENTIFIER
DisplayName        NVARCHAR(200)
Email              NVARCHAR(320)
JobTitle           NVARCHAR(150)
IsActive           BIT
CreatedAtUtc       DATETIME2(3)
UpdatedAtUtc       DATETIME2(3)
RowVersion         ROWVERSION
```

Important uniqueness:

EXAMPLE / FLOW

```
TenantId + EntraObjectId
TenantId + Email
```

Indexes:

SQL

```sql
CREATE UNIQUE INDEX UX_AppUser_Tenant_EntraObject
ON core.AppUser(TenantId, EntraObjectId);

CREATE UNIQUE INDEX UX_AppUser_Tenant_Email
ON core.AppUser(TenantId, Email);
```

Future endpoint mappings:

EXAMPLE / FLOW

```
GET /api/users
→ search AppUser

GET /api/users/{id}
→ AppUser by PK

GET /api/users/me
→ AppUser by Tenant + EntraObjectId
```

✦ constituent.Constituent

Purpose:

> Donor, prospect, company, foundation, volunteer, etc.

Columns:

EXAMPLE / FLOW

```
ConstituentId
TenantId
ConstituentType
DisplayName
FirstName
MiddleName
LastName
OrganizationName
PrimaryEmail
PrimaryPhone
Status
IsAnonymous
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Important checks:

EXAMPLE / FLOW

```
ConstituentType valid values

Status valid values
```

Main query patterns:

EXAMPLE / FLOW

```
Tenant + ConstituentId

Tenant + DisplayName

Tenant + PrimaryEmail

Tenant + Status
```

Start with indexes such as:

```sql
CREATE INDEX IX_Constituent_Tenant_DisplayName
ON constituent.Constituent
(
    TenantId,
    DisplayName
);

CREATE INDEX IX_Constituent_Tenant_Status
ON constituent.Constituent
(
    TenantId,
    Status
);
```

Do **not** create an index for every column.

Every extra index:

```
uses disk
uses memory
must be maintained during INSERT
must be maintained during UPDATE
must be maintained during DELETE
```

Future API mapping:

```
POST /api/constituents
→ constituent.usp_CreateConstituent

GET /api/constituents/{id}
→ SELECT / reporting view

PUT /api/constituents/{id}
→ constituent.usp_UpdateConstituent

GET /api/constituents
→ constituent.usp_SearchConstituents
```

✦ constituent.Address

Columns:

EXAMPLE / FLOW

```
AddressId
TenantId
AddressLine1
AddressLine2
City
StateProvince
PostalCode
CountryCode
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Useful query pattern:

EXAMPLE / FLOW

```
TenantId + PostalCode
```

only add an index if address searching proves important.

✦ constituent.ConstituentAddress

Purpose:

> Link Constituent to Address with business meaning.

Columns:

EXAMPLE / FLOW

```
ConstituentAddressId
TenantId
ConstituentId
AddressId
AddressType
IsPrimary
ValidFrom
ValidTo
CreatedAtUtc
UpdatedAtUtc
```

Important constraints:

EXAMPLE / FLOW

```
ValidTo >= ValidFrom
```

Useful unique filtered index:

```sql
CREATE UNIQUE INDEX UX_ConstituentAddress_Primary
ON constituent.ConstituentAddress
(
    TenantId,
    ConstituentId
)
WHERE IsPrimary = 1;
```

Why is this good?

Without it:

```
John Smith
→ Address A primary
→ Address B primary
```

could accidentally happen.

The filtered unique index lets SQL Server enforce:

```
At most one primary address per constituent.
```

This is a genuine use of a:

```
filtered unique index.
```

✦ fundraising.Campaign

Columns:

```
CampaignId
TenantId
CampaignCode
CampaignName
Description
GoalAmount
StartDate
EndDate
CreatedByUserId
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Unique:

EXAMPLE / FLOW

```
TenantId + CampaignCode
```

Checks:

EXAMPLE / FLOW

```
GoalAmount >= 0

EndDate >= StartDate
```

Useful index:

SQL

```
CREATE UNIQUE INDEX UX_Campaign_Tenant_Code
ON fundraising.Campaign(TenantId, CampaignCode);
```

Future API:

EXAMPLE / FLOW

```
GET /api/campaigns
POST /api/campaigns
GET /api/campaigns/{id}
PUT /api/campaigns/{id}

GET /api/campaigns/{id}/performance
GET /api/campaigns/{id}/history
```

✦ fundraising.Fund

Columns:

EXAMPLE / FLOW

```
FundId
TenantId
FundCode
FundName
Description
IsRestricted
StartDate
EndDate
CreatedByUserId
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Unique:

EXAMPLE / FLOW

```
TenantId + FundCode
```

Useful queries:

EXAMPLE / FLOW

```
active Funds

restricted Funds

Fund giving totals
```

✦ fundraising.Gift

This is one of your most important OLTP tables.

Columns:

EXAMPLE / FLOW

```
GiftId
TenantId
ConstituentId
CampaignId
GiftDate
TotalAmount
CurrencyCode
GiftType
ReferenceNumber
Notes
CreatedByUserId
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Constraints:

EXAMPLE / FLOW

```
TotalAmount > 0
```

Indexes should support actual endpoints.

Typical endpoint:

EXAMPLE / FLOW

```
GET /api/constituents/{id}/gifts
```

Needs:

EXAMPLE / FLOW

```
TenantId
ConstituentId
GiftDate
```

Index:

SQL

```sql
CREATE INDEX IX_Gift_Tenant_Constituent_Date
ON fundraising.Gift
(
    TenantId,
    ConstituentId,
    GiftDate DESC,
    GiftId DESC
)
INCLUDE
(
    CampaignId,
    TotalAmount,
    CurrencyCode,
    GiftType
);
```

Why?

Without a useful index SQL Server may need to inspect much more of:

EXAMPLE / FLOW

```
fundraising.Gift
```

to find one donor's records.

With this index it may be able to navigate directly to:

EXAMPLE / FLOW

```
Tenant 1
+
Constituent 1001
```

and read Gifts already arranged for common date ordering.

`INCLUDE` can provide output columns at the index leaf level without making them part of the ordering key.

Do not add every output field to INCLUDE blindly.

✦ Campaign Gift Index

Endpoint:

EXAMPLE / FLOW

```
GET /api/campaigns/{id}/gifts
```

candidate:

SQL

```
CREATE INDEX IX_Gift_Tenant_Campaign_Date
ON fundraising.Gift
(
    TenantId,
    CampaignId,
    GiftDate DESC,
    GiftId DESC
)
INCLUDE
(
    ConstituentId,
    TotalAmount,
    CurrencyCode
);
```

✦ Gift Date Reporting

If you frequently query:

EXAMPLE / FLOW

```
WHERE TenantId = @TenantId
AND GiftDate >= @FromDate
AND GiftDate < @ToDate
```

an index beginning:

EXAMPLE / FLOW

```
TenantId, GiftDate
```

may be useful.

But do not create all candidate indexes immediately.

Create:

EXAMPLE / FLOW

```
query
→ measure
→ inspect plan
→ index
→ measure again
```

✦ fundraising.GiftAllocation

Columns:

EXAMPLE / FLOW

```
GiftAllocationId
TenantId
GiftId
FundId
AllocationAmount
CreatedAtUtc
UpdatedAtUtc
```

Rules:

EXAMPLE / FLOW

```
AllocationAmount > 0
```

If business says a Fund can only appear once in one Gift:

SQL

```sql
CREATE UNIQUE INDEX UX_GiftAllocation_Gift_Fund
ON fundraising.GiftAllocation
(
    GiftId,
    FundId
);
```

Indexes:

SQL

```sql
CREATE INDEX IX_GiftAllocation_Fund
ON fundraising.GiftAllocation
(
    TenantId,
    FundId,
    GiftId
)
INCLUDE
(
    AllocationAmount
);
```

This supports:

EXAMPLE / FLOW

```
GET /api/funds/{id}/allocations
```

✦ engagement.Interaction

Columns:

EXAMPLE / FLOW

```
InteractionId
TenantId
ConstituentId
FundraiserUserId
InteractionType
InteractionDateUtc
Subject
Notes
CreatedAtUtc
UpdatedAtUtc
```

Typical endpoint:

EXAMPLE / FLOW

```
GET /api/constituents/{id}/interactions
```

Useful index:

SQL

```
CREATE INDEX IX_Interaction_Tenant_Constituent_Date
ON engagement.Interaction
(
    TenantId,
    ConstituentId,
    InteractionDateUtc DESC,
    InteractionId DESC
)
INCLUDE
(
    FundraiserUserId,
    InteractionType,
    Subject
);
```

✦ engagement.Task

Columns:

EXAMPLE / FLOW

```
TaskId
TenantId
ConstituentId
AssignedToUserId
CreatedByUserId
TaskType
Title
Description
DueDateUtc
CompletedAtUtc
Priority
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

One of the best indexes in the project:

SQL

```sql
CREATE INDEX IX_Task_Open_Assignee_Due
ON engagement.Task
(
    TenantId,
    AssignedToUserId,
    DueDateUtc,
    TaskId
)
INCLUDE
(
    ConstituentId,
    Title,
    Priority
)
WHERE CompletedAtUtc IS NULL;
```

Why filtered index?

Most task screens may repeatedly ask:

EXAMPLE / FLOW

```
Show my open Tasks.
```

If the table eventually contains:

EXAMPLE / FLOW

```
10,000,000 historical Tasks
```

but only:

EXAMPLE / FLOW

```
80,000 open Tasks
```

a filtered index containing only the open rows can be much smaller and cheaper to search.

Future endpoint:

EXAMPLE / FLOW

```
GET /api/tasks/open
```

✦ integration.OutboxMessage

Columns:

EXAMPLE / FLOW

```
OutboxMessageId UNIQUEIDENTIFIER
TenantId
MessageType
AggregateType
AggregateId
PayloadJson
OccurredAtUtc
ProcessedAtUtc
RetryCount
LastError
```

Important index:

SQL

```sql
CREATE INDEX IX_Outbox_Unprocessed
ON integration.OutboxMessage
(
    OccurredAtUtc,
    OutboxMessageId
)
INCLUDE
(
    MessageType,
    AggregateType,
    AggregateId,
    RetryCount
)
WHERE ProcessedAtUtc IS NULL;
```

Why?

The worker does not normally need:

EXAMPLE / FLOW

```
all successfully processed historical messages.
```

It repeatedly needs:

EXAMPLE / FLOW

```
only messages waiting to be published.
```

That makes the filtered index very appropriate.

✦ PaymentDb Table Design

PaymentDb needs different performance priorities:

EXAMPLE / FLOW

```
idempotency

fast Payment lookup

provider event lookup

Payment history

attempt lookup

settlement processing

refund validation
```

✦ payment.MerchantAccount

Columns:

EXAMPLE / FLOW

```
MerchantAccountId
TenantId
ProviderName
ExternalMerchantAccountId
DefaultCurrencyCode
IsCardEnabled
IsDirectDebitEnabled
IsWalletEnabled
IsP2PEEnabled
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Unique candidate:

EXAMPLE / FLOW

```
ProviderName + ExternalMerchantAccountId
```

✦ payment.Payment

Columns:

EXAMPLE / FLOW

```
PaymentId
TenantId
MerchantAccountId
FundraisingGiftId
Amount
CurrencyCode
PaymentStatus
ProviderPaymentId
CreatedAtUtc
UpdatedAtUtc
RowVersion
```

Critical index:

SQL

```sql
CREATE INDEX IX_Payment_Tenant_Gift
ON payment.Payment
(
    TenantId,
    FundraisingGiftId
)
INCLUDE
(
    PaymentStatus,
    Amount,
    CurrencyCode,
    ProviderPaymentId,
    CreatedAtUtc
);
```

Future endpoint:

EXAMPLE / FLOW

```
GET /api/gifts/{giftId}/payment
```

Future payment endpoint:

EXAMPLE / FLOW

```
GET /api/payments/{id}
```

uses the PK.

✦ Provider Payment ID

If provider IDs are unique for one provider/merchant, enforce the correct business uniqueness.

Do not merely trust application code.

✦ payment.PaymentMethod

Columns:

EXAMPLE / FLOW

```
PaymentMethodId
TenantId
PaymentMethodType
ProviderToken
ProviderPaymentMethodId
CardBrand
Last4
ExpirationMonth
ExpirationYear
WalletType
CreatedAtUtc
```

Do not place:

EXAMPLE / FLOW

```
PAN
CVV
PIN
raw credentials
```

in this table.

✦ payment.PaymentAttempt

Columns:

EXAMPLE / FLOW

```
PaymentAttemptId
PaymentId
PaymentMethodId
AttemptNumber
ProviderTransactionId
AttemptStatus
AuthorizationCode
FailureCode
FailureMessage
SecureCollectionMode
StartedAtUtc
CompletedAtUtc
```

Unique candidate:

EXAMPLE / FLOW

```
PaymentId + AttemptNumber
```

Index:

SQL

```
CREATE UNIQUE INDEX UX_PaymentAttempt_Number
ON payment.PaymentAttempt
(
    PaymentId,
    AttemptNumber
);
```

Future endpoint:

EXAMPLE / FLOW

```
GET /api/payments/{id}/attempts
```

✦ payment.FraudAssessment

Columns:

EXAMPLE / FLOW

```
FraudAssessmentId
PaymentAttemptId
FraudScore
RiskLevel
Decision
ReasonCodes
EvaluatedAtUtc
```

Index:

EXAMPLE / FLOW

```
PaymentAttemptId
```

because that is the normal relationship path.

✦ payment.Refund

Columns:

EXAMPLE / FLOW

```
RefundId
PaymentId
RefundAmount
RefundReason
RefundStatus
ProviderRefundId
RequestedByUserId
RequestedAtUtc
CompletedAtUtc
RowVersion
```

Important business invariant:

EXAMPLE / FLOW

```
successful refund total
<=
original Payment.Amount
```

A `CHECK` constraint cannot validate a SUM across many Refund rows.

This must be protected by:

EXAMPLE / FLOW

```
transactional procedure
+
appropriate locking
```

Example procedure:

EXAMPLE / FLOW

```
payment.usp_CreateRefund
```

will calculate current successful refunds while protecting the Payment/refund operation from concurrency races.

✦ payment.WebhookEvent

Columns:

EXAMPLE / FLOW

```
WebhookEventId
ProviderName
ProviderEventId
EventType
PayloadJson
SignatureValidated
ReceivedAtUtc
ProcessedAtUtc
ProcessingStatus
FailureMessage
```

Critical unique constraint:

SQL

```sql
CREATE UNIQUE INDEX UX_Webhook_Provider_Event
ON payment.WebhookEvent
(
    ProviderName,
    ProviderEventId
);
```

Why?

Payment providers can retry webhooks.

You must be able to say:

EXAMPLE / FLOW

```
I already received this provider event.
```

This is database-backed idempotency.

✦ Unprocessed Webhook Index

SQL

```sql
CREATE INDEX IX_Webhook_Unprocessed
ON payment.WebhookEvent
(
    ReceivedAtUtc,
    WebhookEventId
)
WHERE ProcessedAtUtc IS NULL;
```

Worker query:

EXAMPLE / FLOW

```
Give me unprocessed events.
```

does not need to scan millions of historical processed events.

✦ payment.Settlement

Columns:

EXAMPLE / FLOW

```
SettlementId
TenantId
MerchantAccountId
ProviderSettlementId
SettlementDate
GrossAmount
FeeAmount
NetAmount
CurrencyCode
SettlementStatus
CreatedAtUtc
```

Unique candidate:

EXAMPLE / FLOW

```
MerchantAccountId + ProviderSettlementId
```

✦ payment.SettlementItem

Columns:

EXAMPLE / FLOW

```
SettlementItemId
SettlementId
PaymentId
GrossAmount
FeeAmount
NetAmount
CreatedAtUtc
```

Indexes:

EXAMPLE / FLOW

```
SettlementId
PaymentId
```

because reconciliation works in both directions:

EXAMPLE / FLOW

```
Which Payments belong to Settlement?
```

and:

EXAMPLE / FLOW

```
Which Settlement contains Payment?
```

✦ payment.IdempotencyRequest

One of the most important technical tables.

Columns:

EXAMPLE / FLOW

```
IdempotencyRequestId
TenantId
IdempotencyKey
RequestHash
PaymentId
ResponseStatusCode
CreatedAtUtc
ExpiresAtUtc
```

Critical index:

SQL

```sql
CREATE UNIQUE INDEX UX_Idempotency_Tenant_Key
ON payment.IdempotencyRequest
(
    TenantId,
    IdempotencyKey
);
```

This is essential because two concurrent requests can both do:

EXAMPLE / FLOW

```
Check if key exists
→ No

Create Payment
```

Application-only checking is not enough.

The database unique constraint becomes the final protection.

Future endpoint:

EXAMPLE / FLOW

```
POST /api/payments
```

will send an idempotency key.

✦ ReportingDb Design

ReportingDb has different goals than OLTP.

FundraisingDb and PaymentDb need:

EXAMPLE / FLOW

```
fast point lookup

fast small writes

short transactions

concurrency
```

ReportingDb needs:

EXAMPLE / FLOW

```
large scans

aggregation

historical analysis

grouping

BI
```

Do not apply the exact same indexing strategy to both.

✦ Dimensions

EXAMPLE / FLOW

```
DimTenant
DimConstituent
DimCampaign
DimFund
DimDate
```

These describe:

EXAMPLE / FLOW

```
Who?
What?
Where?
When?
```

✦ Facts

EXAMPLE / FLOW

```
FactGift
FactPayment
FactInteraction
```

These store measurable business events.

✦ Slowly Changing Dimensions

Your dimension tables contain:

EXAMPLE / FLOW

```
EffectiveFrom
EffectiveTo
IsCurrent
```

This allows Type-2-style history.

Example:

EXAMPLE / FLOW

```
John Smith
City = Richmond
Valid until 2027
```

later:

EXAMPLE / FLOW

```
John Smith
City = Charlotte
current version
```

Reporting can still correctly understand older Gifts under the historical dimension version if that is the warehouse requirement.

✦ FactGift Grain

Define grain before indexes or ETL.

Recommended grain:

> One FactGift row per Gift allocation.

Example:

EXAMPLE / FLOW

```
Gift $1,000

Fund A $700
Fund B $300
```

becomes:

EXAMPLE / FLOW

```
Fact row 1
Gift 10000
Fund A
700

Fact row 2
Gift 10000
Fund B
300
```

Why?

Because then:

SQL

```
SUM(GiftAmount)
GROUP BY Fund
```

naturally answers Fund reporting.

✦ Columnstore for Reporting Facts

Do not use columnstore merely because it exists.

It is most appropriate when fact tables become large and analytical queries scan many rows but relatively few columns.

Candidates:

EXAMPLE / FLOW

```
FactGift
FactPayment
FactInteraction
```

For large warehouse facts, a clustered columnstore index can dramatically reduce storage and improve scan/aggregate workloads.

Do not automatically place columnstore on your small development tables.

Learning sequence:

EXAMPLE / FLOW

```
Start with rowstore
    ↓
Load meaningful volume
    ↓
Measure analytical queries
    ↓
Create columnstore
    ↓
Compare
```

✦ Partitioning

Partitioning is useful primarily for:

EXAMPLE / FLOW

```
manageability

data lifecycle

very large tables

partition elimination in suitable queries
```

It is not a universal "make query fast" switch.

Candidate later:

EXAMPLE / FLOW

```
FactGift by date
FactPayment by date
```

Example partition key:

EXAMPLE / FLOW

```
GiftDateKey
```

Do not partition a table containing 5,000 rows just to say you used partitioning.

Use this as an advanced lab once you have:

EXAMPLE / FLOW

```
millions of fact rows.
```

✦ CRUD Strategy

You said you want to practice all CRUD yourself.

That is exactly what you should do before relying heavily on EF Core.

For each operational entity, manually write:

EXAMPLE / FLOW

```
CREATE
READ
UPDATE
DELETE
SEARCH/LIST
```

✦ Example — Constituent CRUD

CREATE

Future endpoint:

EXAMPLE / FLOW

```
POST /api/constituents
```

SQL skill:

SQL

```
INSERT INTO constituent.Constituent
(
    TenantId,
    ConstituentType,
    DisplayName,
    FirstName,
    LastName,
    PrimaryEmail,
    Status,
    IsAnonymous,
    CreatedAtUtc
)
VALUES
(
    @TenantId,
    @ConstituentType,
    @DisplayName,
    @FirstName,
    @LastName,
    @PrimaryEmail,
    @Status,
    @IsAnonymous,
    SYSUTCDATETIME()
);
```

Learn:

EXAMPLE / FLOW

```
INSERT
parameters
defaults
constraints
identity
```

✦ READ by ID

SQL

```
SELECT
    ConstituentId,
    TenantId,
    ConstituentType,
    DisplayName,
    FirstName,
    LastName,
    PrimaryEmail,
    PrimaryPhone,
    Status,
    IsAnonymous,
    CreatedAtUtc,
    UpdatedAtUtc,
    RowVersion
FROM constituent.Constituent
WHERE TenantId = @TenantId
  AND ConstituentId = @ConstituentId;
```

Always include Tenant boundaries in business queries where appropriate.

✦ UPDATE

SQL

```
UPDATE constituent.Constituent
SET
    DisplayName = @DisplayName,
    PrimaryEmail = @PrimaryEmail,
    PrimaryPhone = @PrimaryPhone,
    Status = @Status,
    UpdatedAtUtc = SYSUTCDATETIME()
WHERE TenantId = @TenantId
  AND ConstituentId = @ConstituentId
  AND RowVersion = @OriginalRowVersion;
```

Then check:

EXAMPLE / FLOW

```
@@ROWCOUNT
```

If zero:

EXAMPLE / FLOW

```
record missing
or
concurrency conflict.
```

.NET will later distinguish those cases.

✦ DELETE

Before implementing physical deletes everywhere, ask:

EXAMPLE / FLOW

```
Should this business entity actually disappear?
```

Examples:

EXAMPLE / FLOW

```
Constituent
Gift
Payment
Settlement
```

often should not casually disappear.

For learning:

EXAMPLE / FLOW

```
practice DELETE
```

but production design may instead use:

EXAMPLE / FLOW

```
status
archive
cancellation
```

for important financial/business history.

Use physical DELETE mainly where the business genuinely permits it.

✦ List/Search

Future endpoint:

EXAMPLE / FLOW

```
GET /api/constituents?search=john&status=1&pageSize=50
```

SQL:

```sql
SELECT ...
FROM constituent.Constituent
WHERE TenantId = @TenantId
  AND (@Status IS NULL OR Status = @Status)
  AND
  (
      @Search IS NULL
      OR DisplayName LIKE @Search + N'%'
  )
ORDER BY DisplayName, ConstituentId;
```

Notice:

```
LIKE @Search + '%'
```

can often use an appropriate index more effectively than:

```
LIKE '%' + @Search + '%'
```

because the latter usually cannot seek to the beginning of a B-tree search range.

✦ CRUD Objects You Should Build

For learning, create these procedures progressively.

FundraisingDb

```
core.usp_CreateTenant
core.usp_UpdateTenant

core.usp_CreateAppUser
core.usp_UpdateAppUser
core.usp_GetAppUser

constituent.usp_CreateConstituent
constituent.usp_UpdateConstituent
constituent.usp_GetConstituent
constituent.usp_SearchConstituents

constituent.usp_AddAddress
constituent.usp_UpdateAddress
constituent.usp_SetPrimaryAddress

fundraising.usp_CreateCampaign
fundraising.usp_UpdateCampaign
fundraising.usp_SearchCampaigns
```

```
fundraising.usp_CreateFund
fundraising.usp_UpdateFund
fundraising.usp_SearchFunds

fundraising.usp_CreateGift
fundraising.usp_UpdateGift
fundraising.usp_GetGift
fundraising.usp_SearchGifts

engagement.usp_CreateInteraction
engagement.usp_GetConstituentInteractions

engagement.usp_CreateTask
engagement.usp_UpdateTask
engagement.usp_CompleteTask
engagement.usp_GetOpenTasks
```

You do not necessarily need a stored procedure for every simple read forever.

The purpose of creating many during the learning phase is to understand:

EXAMPLE / FLOW

```
parameters
result sets
transactions
permissions
plans
error handling
```

Later the .NET architecture can decide when to use:

EXAMPLE / FLOW

```
EF Core
Dapper
stored procedures
```

based on the use case.

✦ PaymentDb Procedures

EXAMPLE / FLOW

```
payment.usp_CreatePayment

payment.usp_GetPayment

payment.usp_StartPaymentAttempt

payment.usp_CompletePaymentAttempt

payment.usp_RecordFraudAssessment

payment.usp_CreateRefund

payment.usp_RecordWebhook

payment.usp_MarkWebhookProcessed

payment.usp_CreateSettlement

payment.usp_AddSettlementItem

payment.usp_GetSettlement

payment.usp_BeginIdempotentRequest
```

✦ ReportingDb Procedures

Reporting endpoints are read-oriented.

Examples:

EXAMPLE / FLOW

```
reporting.usp_GetExecutiveSummary

reporting.usp_GetMonthlyGiving

reporting.usp_GetCampaignPerformance

reporting.usp_GetFundPerformance

reporting.usp_GetTopDonors

reporting.usp_GetPaymentPerformance

reporting.usp_GetRefundSummary

reporting.usp_GetInteractionActivity
```

✦ Stored Procedure Design

A good procedure should have:

EXAMPLE / FLOW

```
clear responsibility

explicit parameters

SET NOCOUNT ON

appropriate transaction handling

no unnecessary SELECT *

predictable result set

good error behavior

security-friendly permissions
```

Example:

SQL

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetGift
    @TenantId BIGINT,
    @GiftId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        g.GiftId,
        g.GiftDate,
        g.TotalAmount,
        g.CurrencyCode,
        g.GiftType,
        c.ConstituentId,
        c.DisplayName,
        ca.CampaignId,
        ca.CampaignName,
        g.RowVersion
    FROM fundraising.Gift AS g
    INNER JOIN constituent.Constituent AS c
        ON c.ConstituentId = g.ConstituentId
       AND c.TenantId = g.TenantId
    LEFT JOIN fundraising.Campaign AS ca
        ON ca.CampaignId = g.CampaignId
       AND ca.TenantId = g.TenantId
    WHERE g.TenantId = @TenantId
      AND g.GiftId = @GiftId;
END;
```

✦ Complex Transaction — Create Gift

This is the most important FundraisingDb procedure.

Future endpoint:

EXAMPLE / FLOW

```
POST /api/gifts
```

One logical operation needs:

EXAMPLE / FLOW

```
Insert Gift

Insert one or more GiftAllocations

Validate allocation total

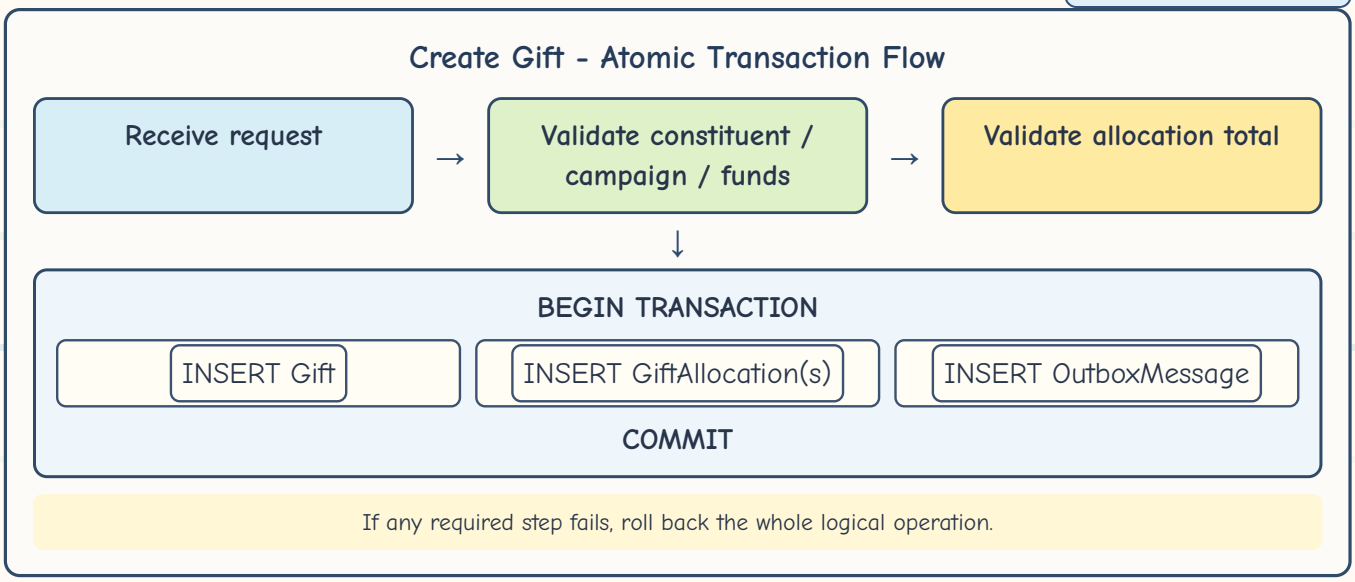
Insert OutboxMessage
```

This must be atomic.

✦ Create Gift Flow

EXAMPLE / FLOW

```
Receive Gift request
      |
      v
Validate Constituent
      |
      v
Validate Campaign
      |
      v
Validate Funds
      |
      v
Validate allocations total
      |
      v
BEGIN TRANSACTION
      |
      +---- INSERT Gift
      |
      +---- INSERT GiftAllocation(s)
      |
      +---- INSERT OutboxMessage
      |
      v
COMMIT
```



If anything fails:

EXAMPLE / FLOW

```
ROLLBACK everything.
```

✦ Transaction Pattern

Use a robust pattern:

SQL

```sql
SET XACT_ABORT ON;

BEGIN TRY
    BEGIN TRANSACTION;

    -- database work

    COMMIT;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0
        ROLLBACK;

    THROW;
END CATCH;
```

This is one of the most important patterns you should know.

✦ Allocation Validation

Suppose request says:

EXAMPLE / FLOW

```
Gift = $1,000

Allocations:
$700
$200
```

Total allocation:

EXAMPLE / FLOW

```
$900
```

Do not commit.

Procedure should validate:

EXAMPLE / FLOW

```
SUM(AllocationAmount)
=
TotalAmount
```

before final commit.

✦ JSON Input for Gift Allocations

Instead of accepting:

EXAMPLE / FLOW

```
@Fund1
@Amount1
@Fund2
@Amount2
...
```

accept an allocation array.

For learning:

SQL

```
@AllocationsJson NVARCHAR(MAX)
```

Example:

```json
[
  {
    "fundId": 300,
    "amount": 700
  },
  {
    "fundId": 301,
    "amount": 300
  }
]
```

Parse:

```sql
SELECT
    FundId,
    AllocationAmount
FROM OPENJSON(@AllocationsJson)
WITH
(
    FundId BIGINT '$.fundId',
    AllocationAmount DECIMAL(19,4) '$.amount'
);
```

This lets you learn:

```
JSON
OPENJSON
table-shaped input
transactions
validation
```

✦ SQL Server 2025 JSON

SQL Server 2025 provides a native `json` type in addition to the traditional `nvarchar` storage pattern. The native type stores JSON in an optimized binary representation and can improve reads and targeted modifications.

For your project:

```
Core business data
→ relational columns

Integration payloads
→ JSON
```

Do not put:

EXAMPLE / FLOW

```
Gift
Campaign
Constituent
Payment
```

entirely inside JSON.

✦ JSON Indexing

If you query an Outbox/Webhook JSON property frequently, you have options.

Traditional approach:

EXAMPLE / FLOW

```
computed column
+
normal index
```

Example:

SQL

```
ALTER TABLE integration.OutboxMessage
ADD GiftIdFromJson AS
    TRY_CONVERT
    (
        BIGINT,
        JSON_VALUE(PayloadJson, '$.giftId')
    );
```

Then:

SQL

```
CREATE INDEX IX_Outbox_GiftIdFromJson
ON integration.OutboxMessage(GiftIdFromJson);
```

SQL Server 2025 also introduces `CREATE JSON INDEX`; Microsoft currently documents JSON indexing for SQL Server 2025.

For your first implementation, prefer explicit relational columns such as:

EXAMPLE / FLOW

```
AggregateId
MessageType
```

for frequently queried technical metadata.

Keep JSON for the variable payload.

✦ XML Lab

Use XML where an external integration actually requires XML.

Example future endpoint:

EXAMPLE / FLOW

```
POST /api/integrations/legacy/gifts/xml
```

SQL procedure:

EXAMPLE / FLOW

```
integration.usp_ImportGiftXml
```

Input:

SQL

```
@GiftXml XML
```

Practice:

EXAMPLE / FLOW

```
.query()
.value()
.exist()
.nodes()
.modify()
```

Example:

SQL

```
SELECT
    X.Gift.value('(ConstituentId/text())[1]', 'BIGINT'),
    X.Gift.value('(TotalAmount/text())[1]', 'DECIMAL(19,4)')
FROM @GiftXml.nodes('/Gift') AS X(Gift);
```

Do not create an XML table for every normal entity.

✦ Temporal Table

Best project candidate:

EXAMPLE / FLOW

```
fundraising.Campaign
```

Why?

Campaign configuration changes over time.

You may want:

EXAMPLE / FLOW

```
What was the GoalAmount three months ago?
```

Add:

EXAMPLE / FLOW

```
SysStartTime
SysEndTime
```

and:

EXAMPLE / FLOW

```
PERIOD FOR SYSTEM_TIME
```

then enable:

EXAMPLE / FLOW

```
SYSTEM_VERSIONING = ON
```

Future endpoints:

EXAMPLE / FLOW

```
GET /api/campaigns/{id}/history

GET /api/campaigns/{id}/as-of?at=...
```

Queries:

SQL

```sql
SELECT *
FROM fundraising.Campaign
FOR SYSTEM_TIME ALL
WHERE CampaignId = @CampaignId;
```

and:

```sql
SELECT *
FROM fundraising.Campaign
FOR SYSTEM_TIME AS OF @AtUtc
WHERE CampaignId = @CampaignId;
```

Remember:

```
CreatedAtUtc / UpdatedAtUtc
```

do not replace temporal history.

They tell you current-row timestamps.

Temporal history preserves old row values.

✦ Views

Views should represent useful reusable read models.

Do not create:

```
vw_Gift
```

that simply does:

```sql
SELECT * FROM Gift;
```

with no benefit.

Create meaningful views.

✦ constituent.vw_ConstituentSummary

Could include:

EXAMPLE / FLOW

```
Constituent

Lifetime Gift amount

Gift count

Last Gift date

Interaction count

Open Task count
```

Useful for:

EXAMPLE / FLOW

```
GET /api/constituents/{id}/summary
```

✦ fundraising.vw_GiftDetail

Join:

EXAMPLE / FLOW

```
Gift
Constituent
Campaign
CreatedBy AppUser
```

Useful for:

EXAMPLE / FLOW

```
Gift detail
reports
```

✦ fundraising.vw_CampaignPerformance

Could expose:

EXAMPLE / FLOW

```
CampaignId
CampaignName
GoalAmount
AmountRaised
GiftCount
DonorCount
RemainingAmount
ProgressPercent
```

✦ Views and Performance

A normal view does not automatically store/copy rows.

It stores a query definition.

Therefore:

EXAMPLE / FLOW

```
View
≠ cache
```

SQL Server still has to execute the underlying logic.

Indexed views exist, but they have strict requirements and should be used only when they genuinely help a measured workload.

✦ Functions

Use functions intentionally.

✦ Scalar UDF

A safe educational example:

EXAMPLE / FLOW

```
fundraising.ufn_CalculateProgressPercent
```

Input:

EXAMPLE / FLOW

```
RaisedAmount
GoalAmount
```

Output:

EXAMPLE / FLOW

```
Progress %
```

Avoid designing scalar UDFs that repeatedly query large tables one row at a time.

✦ Inline Table-Valued Function

Example:

EXAMPLE / FLOW

```
fundraising.ufn_ConstituentGifts
```

Input:

EXAMPLE / FLOW

```
TenantId
ConstituentId
```

Output:

EXAMPLE / FLOW

```
Gift rows
```

An inline TVF behaves much more like a parameterized query expression and is generally more optimizer-friendly than a complex multi-statement TVF.

✦ Multi-Statement TVF

Create one only as a learning exercise.

Then compare it with:

EXAMPLE / FLOW

```
inline TVF
```

using:

EXAMPLE / FLOW

```
actual execution plan
STATISTICS IO
STATISTICS TIME
```

The lesson should be:

> A supported SQL feature is not automatically the best solution for every workload.

✦ Joins

Use your schema to learn every important join type.

INNER JOIN

Example:

EXAMPLE / FLOW

```
Gift
+
Constituent
```

Only Gifts having matching Constituents should exist because of FK integrity.

SQL

```sql
SELECT
    g.GiftId,
    c.DisplayName,
    g.TotalAmount
FROM fundraising.Gift AS g
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId
   AND c.TenantId = g.TenantId;
```

✦ LEFT JOIN

Example:

EXAMPLE / FLOW

```
Gift
LEFT JOIN Campaign
```

because:

EXAMPLE / FLOW

```
CampaignId may be NULL.
```

✦ Many-to-Many Join

EXAMPLE / FLOW

```
Gift
→ GiftAllocation
→ Fund
```

✦ APPLY

Use `CROSS APPLY` / `OUTER APPLY` when a table-valued expression needs to be evaluated relative to an outer row.

Good project examples:

EXAMPLE / FLOW

```
Latest Gift per Constituent

JSON allocations

Inline TVF invocation
```

✦ Filtering and Sorting

Practice:

EXAMPLE / FLOW

```
WHERE
AND
OR
IN
BETWEEN
LIKE
IS NULL
ORDER BY
```

using business endpoints.

Examples:

EXAMPLE / FLOW

```
Active Constituents

Gifts between dates

Campaigns ending this month

Overdue Tasks

Failed Payments

Refunds awaiting completion
```

✦ SARGability

This becomes critical for performance.

Bad:

SQL

```sql
WHERE YEAR(GiftDate) = 2026;
```

Better:

SQL

```sql
WHERE GiftDate >= '20260101'
  AND GiftDate <  '20270101';
```

Why?

The second predicate directly expresses a searchable range on:

EXAMPLE / FLOW

```
GiftDate
```

and gives the optimizer a better chance to use an index seek/range access.

✦ Another Bad Pattern

SQL

```sql
WHERE UPPER(PrimaryEmail) = UPPER(@Email);
```

Applying a function to the indexed column can make searching less efficient.

Better design:

EXAMPLE / FLOW

```
appropriate collation
+
normalized input
+
direct predicate
```

when business semantics allow it.

✦ Aggregates

Use:

EXAMPLE / FLOW

```
SUM
COUNT
AVG
MIN
MAX
```

for real reporting.

Examples:

EXAMPLE / FLOW

```
Lifetime giving

Campaign total

Fund total

Average Gift

Last Gift date

Payment success count
```

✦ GROUP BY

Example:

SQL

```sql
SELECT
    CampaignId,
    SUM(TotalAmount) AS AmountRaised
FROM fundraising.Gift
WHERE TenantId = @TenantId
GROUP BY CampaignId;
```

✦ HAVING

Example:

> Campaigns that have raised more than $100,000.

SQL

```
SELECT
    CampaignId,
    SUM(TotalAmount) AS AmountRaised
FROM fundraising.Gift
WHERE TenantId = @TenantId
GROUP BY CampaignId
HAVING SUM(TotalAmount) > 100000;
```

Difference:

EXAMPLE / FLOW

```
WHERE
→ filter rows before grouping

HAVING
→ filter groups after aggregation
```

✦ Subqueries

Use subqueries for genuine problems.

Example:

> Constituents whose lifetime giving is above average lifetime giving.

Build:

EXAMPLE / FLOW

```
inner aggregate
→ average donor total
→ outer comparison
```

✦ EXISTS

Use when the question is:

> Does a related row exist?

Example:

```sql
SELECT
    c.ConstituentId,
    c.DisplayName
FROM constituent.Constituent AS c
WHERE c.TenantId = @TenantId
AND EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.TenantId = c.TenantId
      AND g.ConstituentId = c.ConstituentId
);
```

✦ NOT EXISTS

Use:

```
EXAMPLE / FLOW
Constituents who have never donated.
```

Prefer `NOT EXISTS` over `NOT IN` when NULL behavior could create surprises.

✦ CTE

A CTE is a named query expression.

It is not:

```
EXAMPLE / FLOW
a temporary table
```

and is not guaranteed to be materialized.

Use it for:

```
EXAMPLE / FLOW
readability

stepwise reporting logic

recursive queries
```

✦ Recursive CTE

FundraisingDb has no genuine entity hierarchy in the core model.

Do not invent one simply to practice recursion.

Use a recursive CTE to generate:

EXAMPLE / FLOW

```
months between Campaign.StartDate and Campaign.EndDate
```

then LEFT JOIN Gift totals so months with zero giving still appear.

Example result:

EXAMPLE / FLOW

```
January    20000
February   31000
March          0
April      42000
```

✦ Set Operators

UNION ALL

Perfect project example:

EXAMPLE / FLOW

```
Constituent timeline
```

Combine:

EXAMPLE / FLOW

```
Gift
Interaction
Task
```

Example output:

EXAMPLE / FLOW

```
2026-01-10 Interaction  Phone call
2026-01-15 Gift         $1,000
2026-01-16 Task         Thank-you letter
```

Use:

EXAMPLE / FLOW

```
UNION ALL
```

when you want to preserve all rows.

✦ UNION

Use when you intentionally want duplicate rows removed.

Remember:

EXAMPLE / FLOW

```
duplicate removal has a cost.
```

Do not use `UNION` when you really need `UNION ALL`.

✦ INTERSECT

Example:

EXAMPLE / FLOW

```
Constituents who have:
Gift
AND
Interaction
```

✦ EXCEPT

Example:

EXAMPLE / FLOW

```
All Constituents
EXCEPT
Constituents who have Gifts
```

Compare it with:

EXAMPLE / FLOW

```
NOT EXISTS
```

as a learning exercise.

✦ Window Functions

Window functions are extremely useful because they calculate across related rows without collapsing detail rows.

✦ ROW_NUMBER

Latest Gift per Constituent:

SQL

```
ROW_NUMBER() OVER
(
    PARTITION BY TenantId, ConstituentId
    ORDER BY GiftDate DESC, GiftId DESC
)
```

Always provide a stable tie-breaker such as:

EXAMPLE / FLOW

```
GiftId
```

when ordering values can tie.

✦ RANK

Use for:

EXAMPLE / FLOW

```
Donor giving ranking
```

where ties receive the same rank and gaps occur.

Example:

EXAMPLE / FLOW

```
1
2
2
4
```

✦ DENSE_RANK

Example:

EXAMPLE / FLOW

```
1
2
2
3
```

✦ NTILE

Segment donors:

SQL

```
NTILE(4)
```

Example groups:

EXAMPLE / FLOW

```
Top 25%
Upper-middle
Lower-middle
Bottom 25%
```

Do not treat this as a business scoring system until product requirements define what the segments mean.

✦ SUM OVER

Running Campaign total:

SQL

```
SUM(TotalAmount)
OVER
(
    PARTITION BY TenantId, CampaignId
    ORDER BY GiftDate, GiftId
    ROWS BETWEEN UNBOUNDED PRECEDING
             AND CURRENT ROW
)
```

The explicit `ROWS` frame avoids peer-group surprises that can occur with default ordered aggregate frames.

✦ LAG

Compare current Gift with previous Gift:

```sql
LAG(TotalAmount)
OVER
(
    PARTITION BY TenantId, ConstituentId
    ORDER BY GiftDate, GiftId
)
```

✦ LEAD

Find next scheduled interaction or next event.

✦ FIRST_VALUE / LAST_VALUE

Use carefully with explicit frames when you mean the entire partition.

Especially:

EXAMPLE / FLOW

```
LAST_VALUE
```

can surprise beginners because the default ordered frame may end at the current row/peer group instead of the entire partition.

✦ Temporary Tables

Use `#temp` tables for larger intermediate workloads.

Example reporting procedure:

EXAMPLE / FLOW

```
#DonorTotals
```

Flow:

EXAMPLE / FLOW

```
aggregate 1 million Gifts
      |
      v
#DonorTotals
      |
      v
index #temp if useful
      |
      v
join Tasks / Interactions
      |
      v
final report
```

Temp tables:

EXAMPLE / FLOW

```
live in tempdb
can have indexes
can have statistics
can be altered
```

They are not simply "memory tables."

✦ Table Variables

Use:

EXAMPLE / FLOW

```
@table
```

for smaller/intermediate sets where their scope and behavior suit the operation.

Example:

EXAMPLE / FLOW

```
@RequestedFundIds
```

Compare actual plans against temp tables.

Do not follow a simplistic rule such as:

EXAMPLE / FLOW

```
less than 100 rows always table variable
```

Measure the workload.

Modern SQL Server improves table-variable compilation behavior, but temp tables can still be superior for many larger or complex workloads.

✦ Dynamic SQL

Use dynamic SQL only when the SQL structure itself genuinely varies.

Good example:

EXAMPLE / FLOW

```
administrator/report builder chooses
approved sort column
```

Use:

EXAMPLE / FLOW

```
sp_executesql
```

for values.

Never concatenate user values directly.

Identifiers cannot normally be parameterized like values, so whitelist:

EXAMPLE / FLOW

```
allowed sort columns

allowed directions
```

and use:

EXAMPLE / FLOW

```
QUOTENAME
```

when constructing approved identifiers.

✦ Pagination

For:

EXAMPLE / FLOW

```
GET /api/constituents
GET /api/gifts
GET /api/payments
```

you need pagination.

✦ OFFSET/FETCH

Simple approach:

```sql
ORDER BY GiftDate DESC, GiftId DESC
OFFSET @Offset ROWS
FETCH NEXT @PageSize ROWS ONLY;
```

Easy for:

EXAMPLE / FLOW

```
Page 1
Page 2
Page 3
```

but deep offsets can become expensive because SQL Server may still need to process/skip many rows.

✦ Keyset Pagination

For high-volume timelines, prefer learning keyset pagination.

If last row from previous page is:

EXAMPLE / FLOW

```
GiftDate = 2026-09-10
GiftId = 155000
```

next query uses:

```sql
WHERE
(
    GiftDate < @LastGiftDate
    OR
    (
        GiftDate = @LastGiftDate
        AND GiftId < @LastGiftId
    )
)
ORDER BY GiftDate DESC, GiftId DESC;
```

This can scale much better for deep navigation when supported by the correct index.

For the .NET phase you should implement both so you understand the trade-off.

✦ Performance Goal — Supporting Heavy CRUD Traffic

You mentioned approximately:

EXAMPLE / FLOW

```
1,000 users
```

The number of users alone does not determine SQL Server load.

Important measures are:

EXAMPLE / FLOW

```
concurrent requests

queries per second

rows touched

transaction duration

query complexity

index quality

connection pooling

hardware / Azure SQL tier
```

A system with:

EXAMPLE / FLOW

```
1,000 signed-in users
```

may have:

EXAMPLE / FLOW

```
50 active database requests
```

at one moment.

Another system with:

EXAMPLE / FLOW

```
100 users
```

may generate enormous load.

So build for:

EXAMPLE / FLOW

```
efficient concurrent transactions
```

rather than chasing the number 1,000 by itself.

✦ Performance Principle — Keep Transactions Short

Bad:

EXAMPLE / FLOW

```
BEGIN TRANSACTION

Update Gift

Call payment provider

Wait 3 seconds

Update Task

Wait for user input

COMMIT
```

This holds database resources far too long.

Correct architecture:

EXAMPLE / FLOW

```
external call outside database transaction

then

BEGIN TRANSACTION
perform required database writes
COMMIT quickly
```

For cross-service work use:

EXAMPLE / FLOW

```
Outbox
events
idempotency
```

instead of a long SQL transaction.

✦ Performance Principle — Index the Access Pattern

Do not ask:

> Which columns should have indexes?

Ask:

> Which queries must execute frequently?

Example endpoint:

EXAMPLE / FLOW

```
GET /api/tasks/open
```

query pattern:

EXAMPLE / FLOW

```
Tenant
Assigned User
Open only
Due Date
```

therefore the filtered Task index follows that pattern.

✦ Clustered Index

A clustered index defines the physical row order structure of a rowstore table.

For OLTP tables in this project, a narrow increasing surrogate PK is usually a reasonable default clustered key.

Example:

EXAMPLE / FLOW

```
GiftId BIGINT IDENTITY
```

Benefits:

EXAMPLE / FLOW

```
narrow

monotonically increasing

stable

efficient PK lookup
```

But remember:

EXAMPLE / FLOW

```
PRIMARY KEY
and
CLUSTERED INDEX
```

are related configuration choices, not the same concept.

✦ Nonclustered Index

Contains index keys plus row locator information.

Use it for alternate access paths.

Example:

EXAMPLE / FLOW

```
Gift by Constituent
```

or:

EXAMPLE / FLOW

```
Task by Assigned User
```

✦ Composite Index

Column order matters.

Index:

EXAMPLE / FLOW

```
TenantId,
ConstituentId,
GiftDate
```

is naturally useful for:

EXAMPLE / FLOW

```
TenantId = ?
ConstituentId = ?
GiftDate range
```

It does not behave identically for:

EXAMPLE / FLOW

```
GiftDate only
```

because B-tree navigation depends strongly on the leftmost key structure.

✦ Covering Index

An index covers a query when SQL Server can return everything needed from that index without going elsewhere for additional columns.

Benefit:

EXAMPLE / FLOW

```
can eliminate Key Lookup
```

But covering every query produces massive indexes and expensive writes.

Balance:

EXAMPLE / FLOW

```
read performance
vs
write cost
```

✦ INCLUDE Columns

Good for columns that need to be returned but do not need to define index ordering.

Example:

SQL

```sql
CREATE INDEX ...
ON Gift(TenantId, ConstituentId, GiftDate)
INCLUDE(TotalAmount, CurrencyCode);
```

Included columns:

EXAMPLE / FLOW

```
live at the leaf
do not determine key ordering
```

✦ Filtered Index

Best when a predictable small subset is frequently queried.

Excellent project examples:

EXAMPLE / FLOW

```
Task
WHERE CompletedAtUtc IS NULL

OutboxMessage
WHERE ProcessedAtUtc IS NULL

WebhookEvent
WHERE ProcessedAtUtc IS NULL
```

✦ Index Seek

A seek means SQL Server uses index navigation to target a key/range.

Seek is often efficient, but:

> Seek does not automatically mean fast.

A seek can still return/read a huge range.

Look at:

EXAMPLE / FLOW

```
Actual Rows
Actual Rows Read
```

where available.

✦ Index Scan

A scan reads through a larger index portion.

A scan is not automatically bad.

For:

EXAMPLE / FLOW

```
monthly executive report
that needs 70% of FactPayment
```

scanning may be more efficient than performing thousands of small lookups.

✦ Key Lookup

Example plan:

EXAMPLE / FLOW

```
Nonclustered Index Seek
      |
      v
Key Lookup
      |
      v
fetch missing columns
```

A few lookups may be fine.

Hundreds of thousands may be expensive.

Possible fix:

EXAMPLE / FLOW

```
covering index
```

but only after measuring.

✦ Heap

A heap is a table without a clustered index.

You should learn it, but most core OLTP tables here should not accidentally remain heaps.

A heap can be useful for specialized:

EXAMPLE / FLOW

```
staging/import
```

scenarios.

Do not treat:

EXAMPLE / FLOW

```
heap = always bad
```

as a universal rule.

✦ Fill Factor

Fill factor controls how full SQL Server makes index leaf pages when creating/rebuilding an index.

Do not set:

EXAMPLE / FLOW

```
FILLFACTOR = 70
```

everywhere "to prevent fragmentation."

That wastes space and memory.

Use non-default fill factor only when a measured page-split/workload pattern justifies it.

SQL Server documents `0` and `100` as equivalent full-page fill-factor values.

✦ Fragmentation

Fragmentation can occur as pages change over time.

Do not run:

EXAMPLE / FLOW

```
REBUILD ALL INDEXES
EVERY NIGHT
```

blindly.

Why?

Rebuilding:

EXAMPLE / FLOW

```
uses CPU
uses I/O
uses log
can block
can create more work than it solves
```

Index maintenance should be based on:

EXAMPLE / FLOW

```
size
workload
fragmentation
page density
maintenance window
```

not a simplistic universal percentage rule.

✦ Statistics

Statistics describe data distribution and help the optimizer estimate:

EXAMPLE / FLOW

```
how many rows
```

will flow through operators.

Bad estimates can cause poor decisions such as:

EXAMPLE / FLOW

```
Nested Loops instead of Hash

wrong memory grant

wrong join order
```

Important:

EXAMPLE / FLOW

```
Index exists
```

does not mean:

EXAMPLE / FLOW

```
optimizer will always use it.
```

Optimizer decisions depend partly on estimated row counts.

✦ Query Optimizer

You write:

```sql
SELECT ...
```

declaratively.

You do not normally command:

```
Use Index X
Use Nested Loops
Read table A first
```

The optimizer considers alternatives and selects a cost-based plan.

Avoid hints until you understand why the optimizer is choosing poorly.

✦ Estimated Execution Plan

Shows the optimizer's compiled plan without executing the query.

Useful when:

```
query cannot safely be executed
```

or before deployment.

✦ Actual Execution Plan

Runs the query and adds runtime information to the selected compiled plan.

Compare:

```
Estimated Rows
vs
Actual Rows
```

Large differences can indicate:

EXAMPLE / FLOW

```
statistics problem

parameter sensitivity

correlated predicates

data skew
```

✦ Common Physical Operators

Understand these.

Nested Loops

Good when:

EXAMPLE / FLOW

```
outer input relatively small
+
inner lookup efficient
```

Example:

EXAMPLE / FLOW

```
50 Gifts
→ seek Constituent 50 times
```

✦ Hash Match

Useful for:

EXAMPLE / FLOW

```
larger unsorted equality joins
aggregations
```

Needs memory.

Too-small memory grant can lead to:

EXAMPLE / FLOW

```
tempdb spills.
```

✦ Merge Join

Useful when inputs are appropriately ordered on join columns.

If SQL Server first must perform expensive Sort operations, that changes the trade-off.

✦ Sort

Needed for operations such as:

EXAMPLE / FLOW

```
ORDER BY

some window functions

merge preparation

distinct processing
```

Large sorts can require memory grants and may spill to tempdb.

✦ Cardinality Estimation

Cardinality estimation means:

> How many rows does SQL Server think this operator will produce?

Example:

EXAMPLE / FLOW

```
Estimated Rows = 1
Actual Rows = 500,000
```

That difference can completely change what plan would have been best.

Always inspect row estimates when diagnosing poor plans.

✦ Parameter Sniffing / Parameter Sensitivity

Procedure:

EXAMPLE / FLOW

```
usp_SearchGifts
```

might first compile using:

EXAMPLE / FLOW

```
Tenant A
1,000 Gifts
```

but later execute for:

EXAMPLE / FLOW

```
Tenant B
50,000,000 Gifts.
```

One cached plan may not be ideal for both.

Parameter sniffing itself is normal.

The problem is:

EXAMPLE / FLOW

```
parameter-sensitive workload
```

where different values genuinely need different plans.

SQL Server 2022+ includes Parameter Sensitive Plan optimization for eligible workloads, and SQL Server 2025 extends adaptive handling with Optional Parameter Plan Optimization for optional predicates.

Do not immediately add:

SQL

```
OPTION (RECOMPILE)
```

to every procedure.

Investigate using:

EXAMPLE / FLOW

```
Query Store

actual plan

statistics

data distribution
```

first.

✦ Optional Search Predicates

A common API search procedure uses:

```sql
WHERE
(
    @CampaignId IS NULL
    OR CampaignId = @CampaignId
)
```

This is easy to write but can create very different optimal plans depending on:

```
NULL
vs
specific value.
```

SQL Server 2025 OPPO can help eligible queries under compatibility level 170, but you should still understand the access pattern and test actual plans.

✦ SET STATISTICS IO

Use:

```sql
SET STATISTICS IO ON;
```

then execute the query.

Focus on:

```
logical reads
```

as an important indication of how much page work occurred.

Example learning:

```
before index
120,000 logical reads

after appropriate index
800 logical reads
```

That is much more meaningful than saying:

```
query felt faster.
```

✦ SET STATISTICS TIME

Use:

```sql
SET STATISTICS TIME ON;
```

Measure:

```
CPU time
elapsed time
```

CPU may exceed elapsed time with parallel execution.

✦ Query Store

Use Query Store for:

```
historical query performance

plan regressions

high CPU queries

high duration queries

plan comparisons
```

This becomes particularly valuable after your .NET API exists.

Then:

```
slow endpoint
```

can be traced to:

```
specific SQL query / plan.
```

✦ DMVs

Learn operational DMVs such as categories around:

EXAMPLE / FLOW

```
requests

sessions

locks

waiting tasks

query stats

indexes
```

Examples you should study:

EXAMPLE / FLOW

```
sys.dm_exec_requests

sys.dm_exec_sessions

sys.dm_tran_locks

sys.dm_os_waiting_tasks

sys.dm_db_index_usage_stats
```

Use DMVs for diagnostics, not application business queries.

✦ Concurrency

High-user systems require correctness before raw speed.

Imagine:

EXAMPLE / FLOW

```
User A edits Gift

User B edits same Gift
```

or:

EXAMPLE / FLOW

```
Two workers process same webhook
```

or:

EXAMPLE / FLOW

```
Two refund requests arrive simultaneously.
```

You need deliberate concurrency behavior.

✦ Optimistic Concurrency

Use `rowversion`.

Best for:

EXAMPLE / FLOW

```
Constituent editing
Campaign editing
Task editing
Gift metadata editing
```

Flow:

EXAMPLE / FLOW

```
A reads version 100

B reads version 100

A updates
→ version becomes 101

B updates using version 100
→ 0 rows affected

B receives concurrency conflict
```

This avoids holding a database lock while a human fills out a web form.

✦ Pessimistic Concurrency

Useful where two transactions must not make the same business decision simultaneously.

Important PaymentDb example:

EXAMPLE / FLOW

```
Refund remaining amount.
```

Suppose:

EXAMPLE / FLOW

```
Payment amount
$1,000

Already refunded
$800

Remaining
$200
```

Two simultaneous requests each try to refund:

EXAMPLE / FLOW

```
$200
```

Both must not observe the same remaining balance and succeed.

Inside a short transaction, a locking strategy such as an update lock can be appropriate.

Conceptually:

SQL

```sql
SELECT ...
FROM payment.Payment WITH (UPDLOCK)
WHERE PaymentId = @PaymentId;
```

Then calculate/refund while the transaction controls the decision.

Use this carefully, not globally.

✦ Shared Lock

Protects reads in lock-based isolation.

Multiple compatible readers can generally coexist.

✦ Exclusive Lock

Protects modifications.

Conflicting operations wait.

✦ Update Lock

Used for data that may be updated and helps reduce certain shared-to-exclusive conversion deadlocks.

✦ Intent Locks

Show higher-level intent.

Examples:

EXAMPLE / FLOW

```
IS
IX
IU
```

You normally let SQL Server manage them automatically.

✦ Blocking

Blocking:

EXAMPLE / FLOW

```
Session A holds incompatible lock
Session B waits
```

Some blocking is normal.

The goal is not:

EXAMPLE / FLOW

```
zero blocking forever.
```

The goal is:

EXAMPLE / FLOW

```
short transactions
efficient queries
correct indexes
appropriate isolation
```

✦ Deadlocks

A deadlock is a cycle:

EXAMPLE / FLOW

```
A owns Gift
A waits for Fund

B owns Fund
B waits for Gift
```

SQL Server breaks the cycle by choosing a victim.

Reduce deadlocks by:

EXAMPLE / FLOW

```
access resources in consistent order

keep transactions short

touch fewer rows

use appropriate indexes

retry safe deadlock-victim operations
```

✦ Lock Escalation

SQL Server can convert many fine-grained locks into coarser locking.

Do not automatically disable escalation.

First investigate:

EXAMPLE / FLOW

```
large transactions

unnecessary scans

poor indexing

very large batches
```

✦ Optimized Locking in SQL Server 2025

SQL Server 2025 supports optimized locking, which can reduce lock memory and blocking for concurrent transactions and can reduce certain lock-escalation/deadlock scenarios.

This is useful modern engine behavior, but still design:

EXAMPLE / FLOW

```
short transactions

correct access order

proper indexes
```

because optimized locking does not make transaction design irrelevant.

✦ Isolation Levels

You need practical labs for:

EXAMPLE / FLOW

```
READ UNCOMMITTED

READ COMMITTED

REPEATABLE READ

SNAPSHOT

SERIALIZABLE
```

and:

EXAMPLE / FLOW

```
READ COMMITTED SNAPSHOT
```

✦ READ UNCOMMITTED

Allows dirty reads.

Do not use:

EXAMPLE / FLOW

```
NOLOCK everywhere
```

to fix poor blocking.

For financial/fundraising values this can return inconsistent results.

Use mainly as a learning lab.

✦ READ COMMITTED

Normal SQL Server default isolation.

Prevents dirty reads.

With traditional locking, readers use shared locking.

With RCSI enabled, committed reads use row versions.

✦ REPEATABLE READ

Prevents changes to rows already read for the transaction duration.

More locking.

Useful for understanding consistency trade-offs.

✦ SERIALIZABLE

Strongest standard lock-based isolation.

Can protect key ranges and prevent phantoms.

Use only when business correctness requires it because concurrency cost can be significant.

✦ SNAPSHOT

Provides transaction-level row-versioned consistency.

Useful for some consistent multi-statement read scenarios.

Update conflicts may occur.

✦ RCSI

`READ_COMMITTED_SNAPSHOT` changes READ COMMITTED reads to use row versions.

Benefits can include:

EXAMPLE / FLOW

```
reader/writer blocking reduction
```

but it is not a magic performance option.

Before enabling it, understand:

EXAMPLE / FLOW

```
tempdb/version store effects
application semantics
monitoring
```

For a modern high-concurrency API, RCSI is an important configuration to evaluate in testing.

✦ One Thousand User Performance Plan

For the operational databases, design around these principles.

Keep request transactions under control

Normal CRUD should often complete in:

EXAMPLE / FLOW

```
milliseconds
```

not hold locks while waiting for:

EXAMPLE / FLOW

```
HTTP calls
humans
email
payment provider
Service Bus
```

Index API access paths

Especially:

EXAMPLE / FLOW

```
TenantId

primary IDs

foreign-key joins

date timelines

open-work filters

provider IDs

idempotency keys
```

Avoid SELECT *

Return only what endpoint requires.

Bad:

SQL

```sql
SELECT *
FROM constituent.Constituent;
```

Better:

```sql
SELECT
    ConstituentId,
    DisplayName,
    PrimaryEmail,
    Status
...
```

This reduces:

```
network data
memory
I/O
coupling
```

Paginate collections

Never let:

```
GET /api/gifts
```

return:

```
5 million Gifts.
```

Set limits such as:

```
default 50
maximum 200
```

in the API later.

SQL must support the paging pattern efficiently.

Do not over-index write-heavy tables

For every INSERT Gift, SQL Server must maintain:

EXAMPLE / FLOW

```
clustered index
+
every relevant nonclustered index
```

Ten unnecessary indexes mean ten extra structures requiring maintenance.

More indexes are not always better.

Parameterize everything

Later .NET must send values as parameters.

Benefits:

EXAMPLE / FLOW

```
security
plan reuse
correct typing
```

Do not concatenate input into T-SQL.

Use appropriate SQL data types in parameters

If column is:

EXAMPLE / FLOW

```
NVARCHAR(320)
```

send a matching parameter type.

Avoid unnecessary implicit conversions.

Use batching for large maintenance/import operations

Instead of one transaction modifying:

EXAMPLE / FLOW

```
10 million rows
```

consider controlled batches when atomic all-or-nothing behavior is not required.

This helps:

EXAMPLE / FLOW

```
log growth
locking
blocking
recovery
```

✦ Outbox Pattern

For distributed reliability:

EXAMPLE / FLOW

```
business change
+
outbox event
```

must commit together.

Example:

SQL

```
BEGIN TRANSACTION;

INSERT Gift;

INSERT GiftAllocation;

INSERT OutboxMessage;

COMMIT;
```

Then worker:

EXAMPLE / FLOW

```
reads unprocessed OutboxMessage
publishes to Service Bus later
marks ProcessedAtUtc
```

This prevents:

EXAMPLE / FLOW

```
Gift committed
but event lost.
```

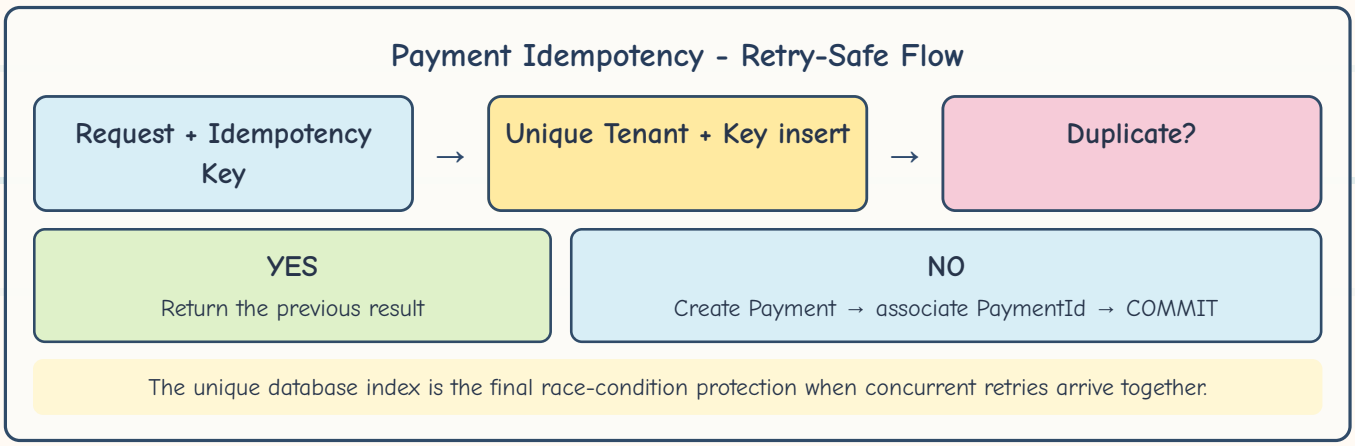
✦ Idempotency Pattern

Payment creation must survive retries.

Process:

EXAMPLE / FLOW

```
Request arrives
       |
       v
Insert IdempotencyRequest
with unique Tenant + Key
       |
       +---- duplicate?
       |       |
       |       v
       |   return previous result
       |
       v
Create Payment
       |
       v
associate PaymentId
       |
       v
COMMIT
```



Unique index provides the race-condition protection.

✦ CDC — Change Data Capture

CDC records source table changes so downstream processes can consume insert/update/delete history.

Potential project use:

EXAMPLE / FLOW

```
FundraisingDb
      |
      v
CDC
      |
      v
Reporting ETL
```

or:

EXAMPLE / FLOW

```
PaymentDb
      |
      v
CDC
      |
      v
ReportingDb
```

Use CDC when downstream systems need change history rather than repeatedly scanning entire operational tables.

Do not confuse CDC with:

EXAMPLE / FLOW

```
application outbox events.
```

Outbox says:

EXAMPLE / FLOW

```
business event happened.
```

CDC says:

EXAMPLE / FLOW

```
database row changed.
```

They solve related but different problems.

✦ Change Tracking

Change Tracking is lighter than CDC.

It mainly tells consumers:

EXAMPLE / FLOW

```
which rows changed
```

rather than preserving the full detailed change history in the same way CDC does.

Good for:

EXAMPLE / FLOW

```
synchronization
```

where consumers already know how to retrieve current state.

Learning comparison:

EXAMPLE / FLOW

```
CDC
→ richer change details

Change Tracking
→ lighter synchronization indicator
```

✦ ETL into ReportingDb

Reporting load should have clear stages.

EXAMPLE / FLOW

```
Extract
      |
      v
Transform
      |
      v
Load Dimensions
      |
      v
Load Facts
```

Do not insert Fact rows before the necessary dimension keys are resolved.

✦ Dimension Load

Example:

EXAMPLE / FLOW

```
Source ConstituentId = 1001
```

Find current:

EXAMPLE / FLOW

```
DimConstituent.ConstituentKey
```

Then FactGift stores:

EXAMPLE / FLOW

```
ConstituentKey
```

rather than relying only on operational IDs.

✦ Incremental Loads

Do not reload:

EXAMPLE / FLOW

```
all Gifts since the beginning of time
```

every five minutes.

Learn incremental techniques using:

EXAMPLE / FLOW

```
CDC

watermark timestamps

source identifiers

event pipelines
```

depending on architecture.

✦ SQL Server Security

The SQL project itself should enforce security.

✦ Login vs User

EXAMPLE / FLOW

```
Login
→ SQL Server instance identity

User
→ database identity
```

Do not run the future API as:

EXAMPLE / FLOW

```
sa
sysadmin
db_owner
```

✦ Custom Database Roles

Create roles such as:

EXAMPLE / FLOW

```
FundraisingApiRole

PaymentApiRole

ReportingApiRole

ReportingReadRole

OperationsRole
```

Grant only what is required.

Microsoft's current SQL Server security guidance recommends role separation and minimum permissions.

✦ GRANT

Example:

SQL

```
GRANT EXECUTE
ON SCHEMA::fundraising
TO FundraisingApiRole;
```

✦ DENY

Explicit prohibition.

Use carefully.

✦ REVOKE

Remove explicit grant/deny.

Do not confuse:

EXAMPLE / FLOW

```
REVOKE
```

with:

EXAMPLE / FLOW

```
DENY.
```

✦ Least Privilege Procedure Model

A strong SQL design for the API can be:

EXAMPLE / FLOW

```
API User

GRANT EXECUTE
on approved procedures

no broad table manipulation
```

This reduces exposed database surface.

You can later compare this approach with direct EF Core table permissions.

✦ Row-Level Security

FundraisingDb is multi-tenant.

RLS can enforce:

EXAMPLE / FLOW

```
Tenant 1
→ only Tenant 1 rows.
```

Use a security predicate function plus Security Policy.

Do this after normal business queries are working because RLS makes troubleshooting more advanced.

RLS is an extra defense layer.

It does not remove the need to:

EXAMPLE / FLOW

```
include TenantId appropriately
validate tenant identity
design authorization correctly
```

✦ Dynamic Data Masking

Good learning fields:

EXAMPLE / FLOW

```
Constituent.PrimaryEmail

Constituent.PrimaryPhone
```

Create a support/reporting role that can query the row but does not need the complete value.

DDM is not encryption.

✦ TDE

Transparent Data Encryption protects database files/backups at rest.

Enable it as an administration/security lab.

Important:

EXAMPLE / FLOW

```
back up certificate/private key.
```

TDE does not stop an authorized SELECT.

Microsoft recommends TDE for at-rest database protection as part of SQL Server security defense in depth.

✦ Always Encrypted

Best advanced lab:

EXAMPLE / FLOW

```
PaymentDb
```

on one learning-only sensitive field/reference.

Understand:

EXAMPLE / FLOW

```
Column Master Key

Column Encryption Key

client driver

deterministic encryption

randomized encryption

secure enclaves
```

Do not encrypt every column.

Always Encrypted with secure enclaves can support richer operations on protected values when properly configured.

✦ TLS

SQL Server should be accessed using encrypted network communication.

TDE:

EXAMPLE / FLOW

```
protects at rest.
```

TLS:

EXAMPLE / FLOW

```
protects in transit.
```

These are different security layers.

✦ SQL Injection

The database project should contain an explicit lab showing:

EXAMPLE / FLOW

```
unsafe string SQL
vs
parameterized SQL
```

Future .NET queries must be parameterized.

Stored procedures are not automatically safe if they concatenate untrusted input into dynamic SQL.

✦ Backup Strategy

Your project is incomplete until you can recover it.

For the OLTP databases, practice:

EXAMPLE / FLOW

```
Full backup

Differential backup

Transaction log backup
```

✦ Recovery Models

Study:

EXAMPLE / FLOW

```
SIMPLE

FULL

BULK_LOGGED
```

For production-like FundraisingDb and PaymentDb exercises, use:

EXAMPLE / FLOW

```
FULL
```

when practicing:

EXAMPLE / FLOW

```
log backups
point-in-time restore
```

ReportingDb may use a different strategy depending on how easily it can be rebuilt, but still needs a deliberate recovery plan.

✦ Full Backup

EXAMPLE / FLOW

```
entire database backup foundation
```

✦ Differential Backup

Contains changed extents since the differential base full backup.

Useful to reduce restore work.

✦ Log Backup

Allows recovery through the transaction-log chain.

Essential for low-RPO point-in-time recovery.

✦ Point-in-Time Recovery Lab

Create this exercise.

EXAMPLE / FLOW

```
09:00
Full backup

10:00
Log backup

10:10
Insert Gifts

10:20
Log backup

10:25
Accidental DELETE

10:26
Discover problem
```

Goal:

EXAMPLE / FLOW

```
restore to 10:24:59.
```

Until you can perform this exercise successfully, you have not really learned backups.

✦ RESTORE VERIFYONLY

Useful backup verification.

But:

EXAMPLE / FLOW

```
VERIFYONLY
≠ full disaster recovery test.
```

Periodically perform a real restore to another database/server and run:

EXAMPLE / FLOW

```
DBCC CHECKDB.
```

✦ DBCC CHECKDB

Run integrity checks on:

EXAMPLE / FLOW

```
FundraisingDb
PaymentDb
ReportingDb
```

It helps detect logical/physical consistency problems.

Do not wait until the database fails before learning how to use it.

✦ Index Maintenance

Do not blindly rebuild everything.

A mature maintenance strategy should consider:

EXAMPLE / FLOW

```
index size

fragmentation

page density

workload

maintenance window

log impact
```

✦ Statistics Maintenance

Auto statistics handle much of normal work.

But large ETL/bulk changes or unusual workloads may require targeted statistics maintenance.

Do not run:

EXAMPLE / FLOW

```
UPDATE STATISTICS every table FULLSCAN
```

every hour without reason.

✦ SQL Agent Jobs

Create learning jobs for:

EXAMPLE / FLOW

```
Full backup

Log backup

CHECKDB

Outbox cleanup/monitoring

Reporting ETL

history cleanup
```

Application workloads should not depend entirely on a person manually opening SSMS.

✦ Outbox Retention

Do not allow:

EXAMPLE / FLOW

```
Processed OutboxMessage
```

to grow forever without policy.

Possible retention:

EXAMPLE / FLOW

```
30 days
90 days
1 year
```

depending on audit/business needs.

Archive/delete in controlled batches.

✦ Webhook Retention

Similarly decide retention for:

EXAMPLE / FLOW

```
WebhookEvent
```

because raw provider payloads can become very large.

Keep what compliance/audit needs require.

✦ Temporal History Retention

Temporal Campaign history also needs:

EXAMPLE / FLOW

```
retention planning.
```

System versioning does not mean:

EXAMPLE / FLOW

```
keep infinite history without cost.
```

✦ High Availability — Availability Groups

After backup/recovery fundamentals, learn:

EXAMPLE / FLOW

```
Primary replica

Secondary replica

Synchronous commit

Asynchronous commit

Listener

Automatic failover

Planned failover

Forced failover
```

Do not confuse:

EXAMPLE / FLOW

```
Availability Group
```

with:

EXAMPLE / FLOW

```
backup.
```

A bad DELETE can replicate.

You still need backups.

✦ Replication

Use as a separate advanced lab.

Possible learning use:

EXAMPLE / FLOW

```
FundraisingDb
      |
      v
Transactional Replication
      |
      v
Reporting/Read database
```

But your final ReportingDb architecture will more naturally teach:

EXAMPLE / FLOW

```
ETL / CDC / events
```

so replication is primarily a concept/lab unless a real requirement appears.

✦ Performance Test Data

You cannot learn SQL Server performance from:

EXAMPLE / FLOW

```
10 Constituents
20 Gifts
```

Generate realistic volumes.

Start:

EXAMPLE / FLOW

```
3 tenants

100 AppUsers

10,000 Constituents

50 Campaigns

50 Funds

100,000 Gifts

200,000 GiftAllocations

300,000 Interactions

100,000 Tasks

50,000 Payments

100,000 PaymentAttempts
```

Then later scale:

EXAMPLE / FLOW

```
1 million Gifts

2 million Allocations

millions of fact rows
```

Now execution-plan differences become visible.

✦ Performance Test Cases

For each high-use procedure, save:

EXAMPLE / FLOW

```
SQL text

parameters

estimated plan

actual plan

STATISTICS IO

STATISTICS TIME

row counts

before/after indexes
```

Example folder:

EXAMPLE / FLOW

```
11-Performance/
    Gifts/
      GetConstituentGifts-Before.sql
      GetConstituentGifts-After.sql
      Notes.md
```

That becomes your actual SQL Server tuning portfolio.

✦ Load Testing Preparation

The actual 1,000-user HTTP load test will happen later through .NET tools.

SQL Server preparation now should ensure:

EXAMPLE / FLOW

```
all list operations paginated

transactions short

queries parameterized

important indexes present

Query Store enabled

rowversion used

idempotency protected

outbox workers query filtered indexes

RCSI evaluated

blocking monitored

procedures return limited data
```

✦ Endpoint-to-SQL Map — Fundraising

Future Endpoint	SQL Responsibility
`POST /api/constituents`	`constituent.usp_CreateConstituent`
`GET /api/constituents/{id}`	view/query by Tenant + ID
`PUT /api/constituents/{id}`	`usp_UpdateConstituent`, RowVersion
`GET /api/constituents`	`usp_SearchConstituents`, pagination
`POST /api/constituents/{id}/addresses`	transactional address/link insert
`GET /api/constituents/{id}/summary`	summary view/procedure
`GET /api/constituents/{id}/timeline`	`UNION ALL` timeline query
`POST /api/campaigns`	`fundraising.usp_CreateCampaign`
`GET /api/campaigns/{id}/performance`	aggregates/window/report view
`GET /api/campaigns/{id}/history`	Temporal `FOR SYSTEM_TIME ALL`
`POST /api/funds`	`fundraising.usp_CreateFund`
`POST /api/gifts`	`fundraising.usp_CreateGift` transaction
`GET /api/gifts/{id}`	Gift detail query/view
`PUT /api/gifts/{id}`	RowVersion update
`GET /api/gifts`	`usp_SearchGifts`, pagination

Future Endpoint	SQL Responsibility
`POST /api/interactions`	insert Interaction
`POST /api/tasks`	insert Task
`PUT /api/tasks/{id}`	RowVersion update
`PUT /api/tasks/{id}/complete`	atomic Task completion
`GET /api/tasks/open`	filtered-index query

✦ Endpoint-to-SQL Map — Payments

Future Endpoint	SQL Responsibility
`POST /api/payments`	idempotency + Payment transaction
`GET /api/payments/{id}`	Payment detail
`GET /api/payments/{id}/attempts`	PaymentAttempt query
`POST /api/payments/{id}/refunds`	concurrency-safe refund transaction
`GET /api/payments/{id}/refunds`	Refund query
`POST /api/payment-provider/webhooks`	insert unique WebhookEvent
`GET /api/settlements`	paginated Settlement query
`GET /api/settlements/{id}`	Settlement + items
`GET /api/admin/webhooks/failed`	filtered failed/unprocessed query

✦ Endpoint-to-SQL Map — Reporting

Future Endpoint	SQL Responsibility
`GET /api/reports/executive`	warehouse aggregate
`GET /api/reports/giving/monthly`	FactGift + DimDate
`GET /api/reports/campaigns`	FactGift + DimCampaign
`GET /api/reports/funds`	FactGift + DimFund
`GET /api/reports/donors/top`	aggregate + ranking
`GET /api/reports/payments/performance`	FactPayment
`GET /api/reports/payments/refunds`	FactPayment aggregates
`GET /api/reports/interactions`	FactInteraction

✦ SQL Server Build Order

Do not jump directly to advanced features.

Build in this sequence.

Stage A — Engine and database fundamentals

Learn/create:

EXAMPLE / FLOW

```
SQL Server instance

SSMS

Database

Data/log files

Schemas

Recovery models

Compatibility level

Query Store
```

✦ Stage B — FundraisingDb relational foundation

Create in this order:

EXAMPLE / FLOW

```
Tenant
AppUser
Constituent
Address
ConstituentAddress
Campaign
Fund
Gift
GiftAllocation
Interaction
Task
OutboxMessage
```

Then add:

EXAMPLE / FLOW

```
PK
FK
UNIQUE
CHECK
DEFAULT
```

Then seed data.

✦ Stage C — Basic SQL on FundraisingDb

Write:

EXAMPLE / FLOW

```
INSERT
SELECT
UPDATE
DELETE

WHERE
ORDER BY
LIKE
IN
BETWEEN
NULL
CASE
functions
```

for every appropriate table.

✦ Stage D — Relational querying

Practice:

EXAMPLE / FLOW

```
INNER JOIN
LEFT JOIN
M:N joins
GROUP BY
HAVING
subqueries
EXISTS
NOT EXISTS
```

Build business reports rather than random examples.

✦ Stage E — Advanced querying

Practice:

EXAMPLE / FLOW

```
CTE
recursive CTE
UNION
UNION ALL
INTERSECT
EXCEPT
APPLY
window functions
```

✦ Stage F — Database programming

Build:

EXAMPLE / FLOW

```
Views

Stored Procedures

Scalar UDF

Inline TVF

Transactions

TRY/CATCH

THROW

XACT_STATE

XACT_ABORT

#temp

@table

dynamic SQL

pagination
```

✦ Stage G — Performance

Only now begin serious performance tuning.

Learn:

EXAMPLE / FLOW clustered index

nonclustered index

composite index

unique index

filtered index

covering index

INCLUDE

heap

fill factor

fragmentation

statistics

estimated plan

actual plan

cardinality estimation

seek

scan

lookup

sort

nested loops

hash

merge join

SARGability

parameter sensitivity

Query Store

STATISTICS IO

STATISTICS TIME

✦ Stage H — PaymentDb

Create all ten Payment tables.

Then implement:

EXAMPLE / FLOW

```
idempotency

Payment creation

attempt workflow

fraud assessments

refund transaction

Webhook deduplication

settlement
```

This database is where your transaction/concurrency skill should become much stronger.

✦ Stage I — Concurrency

Run two-session labs for:

EXAMPLE / FLOW

```
Shared lock

Exclusive lock

Update lock

Blocking

Deadlock

READ COMMITTED

REPEATABLE READ

SERIALIZABLE

SNAPSHOT

RCSI

rowversion

optimistic concurrency

pessimistic concurrency
```

✦ Stage J — Advanced Data Features

Add:

EXAMPLE / FLOW

```
JSON Outbox

JSON Webhook

OPENJSON Gift allocation import

XML legacy integration

Temporal Campaign

CDC

Change Tracking comparison
```

✦ Stage K — ReportingDb

Create:

EXAMPLE / FLOW

```
DimDate

DimTenant

DimConstituent

DimCampaign

DimFund

FactGift

FactPayment

FactInteraction
```

Then build:

EXAMPLE / FLOW

```
ETL

SCD history

star schema queries

columnstore experiment

partitioning experiment

analytical stored procedures
```

✦ Stage L — Security

Implement:

EXAMPLE / FLOW

```
Logins

Users

Roles

GRANT

DENY

REVOKE

Schema permissions

Object permissions

least privilege

RLS

DDM

TDE lab

Always Encrypted lab

TLS configuration concepts
```

✦ Stage M — Administration

Finish with:

EXAMPLE / FLOW SQL Agent

```
Full backup
Differential backup
Log backup
FULL recovery
Point-in-time recovery
Restore testing
DBCC CHECKDB
Statistics maintenance
Index maintenance
Query Store monitoring
Capacity monitoring
Availability Groups
Failover
Replication concepts
```

✦ What Belongs in SQL Server and What Does Not

SQL Server should own:

EXAMPLE / FLOW

```
data integrity
relational constraints
transactional consistency
indexes
stored data
query execution
row concurrency
database security
database recovery
```

.NET later should own:

EXAMPLE / FLOW

```
HTTP

JSON API contract

business orchestration across services

authentication token validation

authorization policies

external payment HTTP calls

Service Bus publishing/consumption

DTO validation

user-facing error responses
```

Angular later should own:

EXAMPLE / FLOW

```
screens

forms

client navigation

UI validation

tables

charts

user experience
```

Azure later should own:

EXAMPLE / FLOW

```
hosting

managed identity

secrets

messaging

scaling

monitoring

networking

containers
```

Do not put everything in SQL Server merely because T-SQL can technically do it.

✦ Final SQL Server Performance Architecture

For the future application workload, aim for this:

EXAMPLE / FLOW

```
.NET API
   |
   | parameterized commands
   v
SQL Server
   |
   +---- Narrow, indexed lookup
   |
   +---- Short transactions
   |
   +---- Optimistic concurrency
   |
   +---- Correct constraints
   |
   +---- Stored procedures for
   |     complex atomic workflows
   |
   +---- Filtered indexes for
   |     operational queues
   |
   +---- Outbox for cross-service work
   |
   +---- Query Store monitoring
   |
   +---- Reporting moved to ReportingDb
```

Avoid:

EXAMPLE / FLOW

```
huge SELECT *

long-running transactions

cross-database business transactions

NOLOCK everywhere

indexes on every column

dynamic SQL concatenation

unbounded list endpoints

application-only uniqueness checks

payment calls inside SQL transactions

reporting scans on OLTP all day

sysadmin application account
```

✦ SQL Server Completion Checklist

Before moving fully to the .NET document, your SQL project should eventually be able to demonstrate all of these.

Database design

EXAMPLE / FLOW

```
✓ Three databases

✓ Correct schemas

✓ 30 business/warehouse tables

✓ Correct data types

✓ PKs

✓ FKs inside database boundaries

✓ Unique business keys

✓ CHECK constraints

✓ Defaults

✓ Normalization in OLTP

✓ Denormalized star schema in OLAP
```

Development SQL

EXAMPLE / FLOW ✓ INSERT

✓ SELECT

✓ UPDATE

✓ DELETE

✓ Filtering

✓ Sorting

✓ Functions

✓ CASE

✓ Aggregates

✓ GROUP BY

✓ HAVING

✓ Joins

✓ Subqueries

✓ EXISTS

✓ NOT EXISTS

✓ CTEs

✓ Recursive CTE

✓ APPLY

✓ Set operators

✓ Window functions

Database programming

EXAMPLE / FLOW

- ✓ Views
- ✓ Stored procedures
- ✓ Scalar UDF
- ✓ Inline TVF
- ✓ Multi-statement TVF comparison
- ✓ Transactions
- ✓ TRY/CATCH
- ✓ THROW
- ✓ XACT_STATE
- ✓ XACT_ABORT
- ✓ Temp tables
- ✓ Table variables
- ✓ Dynamic SQL
- ✓ Pagination

Performance

EXAMPLE / FLOW

- ✓ Clustered indexes
- ✓ Nonclustered indexes
- ✓ Composite indexes
- ✓ Unique indexes
- ✓ Filtered indexes
- ✓ Covering indexes
- ✓ INCLUDE
- ✓ Heap lab
- ✓ Fill-factor understanding
- ✓ Fragmentation
- ✓ Statistics
- ✓ SARGability

- ✓ Estimated plans
- ✓ Actual plans
- ✓ Cardinality estimation
- ✓ Seek
- ✓ Scan
- ✓ Key Lookup
- ✓ Sort
- ✓ Nested Loops
- ✓ Hash Match
- ✓ Merge Join
- ✓ Parameter sensitivity
- ✓ PSP
- ✓ OPPO awareness
- ✓ Query Store
- ✓ STATISTICS IO
- ✓ STATISTICS TIME

Concurrency

EXAMPLE / FLOW

- ✓ Shared lock
- ✓ Exclusive lock
- ✓ Update lock
- ✓ Intent locks
- ✓ Blocking
- ✓ Deadlocks
- ✓ Lock escalation
- ✓ Optimized locking awareness
- ✓ Optimistic concurrency
- ✓ Pessimistic concurrency
- ✓ READ UNCOMMITTED

✓ READ COMMITTED

✓ REPEATABLE READ

✓ SERIALIZABLE

✓ SNAPSHOT

✓ RCSI

Advanced data

EXAMPLE / FLOW

✓ JSON_VALUE

✓ JSON_QUERY

✓ OPENJSON

✓ FOR JSON

✓ native JSON awareness

✓ XML data type

✓ XML query/value/exist/nodes

✓ Temporal table

✓ CDC

✓ Change Tracking

✓ Outbox

OLAP

EXAMPLE / FLOW

- ✓ OLTP vs OLAP
- ✓ ETL
- ✓ Dimension tables
- ✓ Fact tables
- ✓ Grain
- ✓ Surrogate keys
- ✓ Slowly changing dimensions
- ✓ Star schema
- ✓ Columnstore
- ✓ Partitioning

Security

EXAMPLE / FLOW ✓ Authentication concept

- ✓ Login
- ✓ User
- ✓ Server role
- ✓ Database role
- ✓ GRANT
- ✓ DENY
- ✓ REVOKE
- ✓ Schema permission
- ✓ Object permission
- ✓ RLS
- ✓ DDM
- ✓ TDE
- ✓ Always Encrypted concepts
- ✓ TLS
- ✓ SQL injection prevention

✓ Parameterization

✓ Least privilege

Administration

EXAMPLE / FLOW ✓ SIMPLE recovery

✓ FULL recovery

✓ Full backup

✓ Differential backup

✓ Log backup

✓ Restore

✓ NORECOVERY

✓ RECOVERY

✓ Point-in-time recovery

✓ Tail-log concept

✓ CHECKSUM

✓ RESTORE VERIFYONLY

✓ DBCC CHECKDB

✓ SQL Agent

✓ Index maintenance

✓ Statistics maintenance

✓ History retention

✓ Availability Groups

✓ Failover

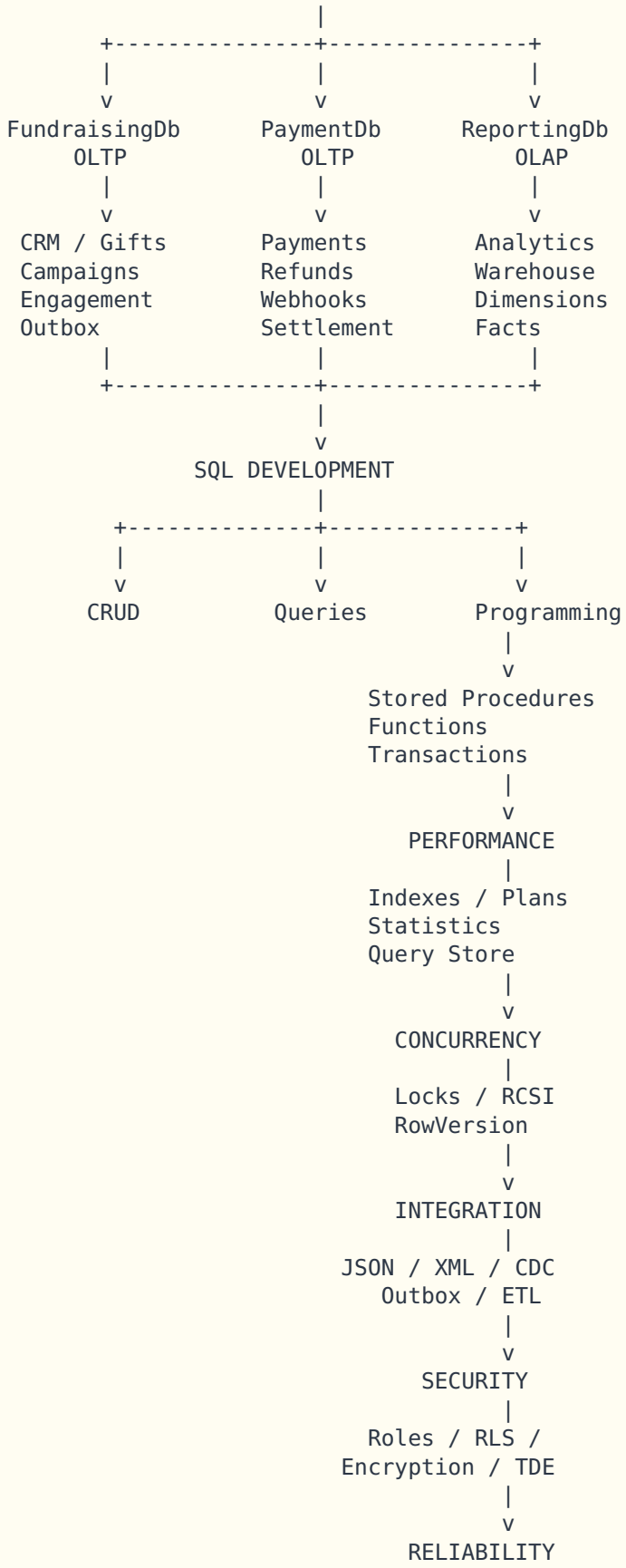
✓ Replication concepts

✦ Final Project Memory Model

The whole SQL Server portion can be remembered like this:

EXAMPLE / FLOW

SQL SERVER PROJECT



```
                    |
Backup / Restore /
CHECKDB / HA / DR
```

The most important development rule for this entire SQL Server project is:

> **Start with correct relational design and business integrity. Then write the CRUD and business queries. Then create transactional stored procedures for operations that must succeed together. After you have realistic data and workload, measure execution plans and I/O before adding indexes. Then add concurrency, integration, security, reporting, and administration features. Performance comes from correct data modeling, selective queries, appropriate indexes, short transactions, good estimates, and measurement—not from adding every advanced SQL feature everywhere.**

And for the future 1,000-user .NET application, the SQL Server side should be prepared around:

EXAMPLE / FLOW

```
short transactions
+
parameterized queries
+
pagination
+
appropriate indexes
+
rowversion
+
idempotency
+
RCSI evaluation
+
Outbox
+
Query Store
+
Reporting workload isolation
+
least privilege
```

That gives you a database foundation that is useful not only for learning SQL syntax, but for the actual `.NET → microservices → Azure → Angular → AI` project that will sit on top of it.